



**ADMM: примеры, солверы. Непрерывные  
методы оптимизации.**

**Даня Меркулов**

ФКН ВШЭ

# ADMM

# ADMM: общая постановка

Задача оптимизации:

$$\min_{x,z} f(x) + r(z)$$

$$\text{при } Ax + Bz = c$$

Здесь  $r(z)$  — второе слагаемое целевой функции (обычно негладкое), а **не** двойственная функция  $g(\nu)$  из прошлых разделов.

# ADMM: общая постановка

Задача оптимизации:

$$\min_{x,z} f(x) + r(z)$$

$$\text{при } Ax + Bz = c$$

Здесь  $r(z)$  — второе слагаемое целевой функции (обычно негладкое), а **не** двойственная функция  $g(\nu)$  из прошлых разделов.

Добавим к целевой функции штраф за нарушение ограничения:

$$\min_{x,z} f(x) + r(z) + \frac{\rho}{2} \|Ax + Bz - c\|^2$$

$$\text{при } Ax + Bz = c$$

# ADMM: общая постановка

Задача оптимизации:

$$\min_{x,z} f(x) + r(z)$$

$$\text{при } Ax + Bz = c$$

Здесь  $r(z)$  — второе слагаемое целевой функции (обычно негладкое), а **не** двойственная функция  $g(\nu)$  из прошлых разделов.

Добавим к целевой функции штраф за нарушение ограничения:

$$\min_{x,z} f(x) + r(z) + \frac{\rho}{2} \|Ax + Bz - c\|^2$$

$$\text{при } Ax + Bz = c$$

где  $\rho > 0$  — параметр. Модифицированная функция Лагранжа:

$$L_\rho(x, z, \nu) = f(x) + r(z) + \nu^T (Ax + Bz - c) + \frac{\rho}{2} \|Ax + Bz - c\|^2$$

## Шаги ADMM (формально)

ADMM повторяет следующие шаги для  $k = 1, 2, 3, \dots$ :

1. Обновить  $x$ :  $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$

## Шаги ADMM (формально)

ADMM повторяет следующие шаги для  $k = 1, 2, 3, \dots$ :

1. Обновить  $x$ :  $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить  $z$ :  $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$

# Шаги ADMM (формально)

**ADMM повторяет следующие шаги для  $k = 1, 2, 3, \dots$ :**

1. Обновить  $x$ :  $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить  $z$ :  $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$
3. Обновить  $\nu$ :  $\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$



# Шаги ADMM (формально)

**ADMM повторяет следующие шаги для  $k = 1, 2, 3, \dots$ :**

1. Обновить  $x$ :  $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить  $z$ :  $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$
3. Обновить  $\nu$ :  $\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$

# Шаги ADMM (формально)

**ADMM повторяет следующие шаги для  $k = 1, 2, 3, \dots$ :**

1. Обновить  $x$ :  $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить  $z$ :  $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$
3. Обновить  $\nu$ :  $\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$

здесь  $\nu$  — множитель ограничения, шаг по нему идёт с коэффициентом  $\rho$ .

## Шаги ADMM (формально)

ADMM повторяет следующие шаги для  $k = 1, 2, 3, \dots$ :

1. Обновить  $x$ :  $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить  $z$ :  $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$
3. Обновить  $\nu$ :  $\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$

здесь  $\nu$  — множитель ограничения, шаг по нему идёт с коэффициентом  $\rho$ .

**Отличие от ALM.** Обычный ALM заменил бы первые два шага совместной минимизацией:

$$(x_k, z_k) = \arg \min_{x, z} L_\rho(x, z, \nu_{k-1})$$

## Шаги ADMM (формально)

ADMM повторяет следующие шаги для  $k = 1, 2, 3, \dots$ :

1. Обновить  $x$ :  $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить  $z$ :  $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$
3. Обновить  $\nu$ :  $\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$

здесь  $\nu$  — множитель ограничения, шаг по нему идёт с коэффициентом  $\rho$ .

**Отличие от ALM.** Обычный ALM заменил бы первые два шага совместной минимизацией:

$$(x_k, z_k) = \arg \min_{x, z} L_\rho(x, z, \nu_{k-1})$$

ADMM **не** минимизирует  $(x, z)$  совместно, иначе переменные снова оказались бы связаны. Один проход по каждой переменной обходится чуть большим числом итераций, зато подзадачи  $f$  и  $r$  решаются по отдельности, часто в замкнутой форме.

## Масштабированная форма: компактные шаги

Введём масштабированный множитель  $u = \nu/\rho$  и невязку  $r = Ax + Bz - c$ . Достроим квадрат:

$$\nu^T r + \frac{\rho}{2} \|r\|^2 = \frac{\rho}{2} \left( \|r\|^2 + \frac{2}{\rho} \nu^T r \right) = \frac{\rho}{2} \left\| r + \frac{\nu}{\rho} \right\|^2 - \frac{1}{2\rho} \|\nu\|^2 = \frac{\rho}{2} \|r + u\|^2 - \underbrace{\frac{\rho}{2} \|u\|^2}_{\text{const по } x, z}.$$

Линейный и квадратичный члены слились в один сдвинутый квадрат, а  $\rho$  ушёл внутрь нормы. Получаем компактную запись (исходный множитель  $\nu = \rho u$ ):

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

## Масштабированная форма: компактные шаги

Введём масштабированный множитель  $u = \nu/\rho$  и невязку  $r = Ax + Bz - c$ . Достроим квадрат:

$$\nu^T r + \frac{\rho}{2} \|r\|^2 = \frac{\rho}{2} \left( \|r\|^2 + \frac{2}{\rho} \nu^T r \right) = \frac{\rho}{2} \left\| r + \frac{\nu}{\rho} \right\|^2 - \frac{1}{2\rho} \|\nu\|^2 = \frac{\rho}{2} \|r + u\|^2 - \underbrace{\frac{\rho}{2} \|u\|^2}_{\text{const по } x, z}.$$

Линейный и квадратичный члены слились в один сдвинутый квадрат, а  $\rho$  ушёл внутрь нормы. Получаем компактную запись (исходный множитель  $\nu = \rho u$ ):

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

В этой форме  $u$ -шаг идёт без  $\rho$ , поскольку её роль уже внутри нормы. Далее используем именно эту форму.

Интуиция: что делают шаги  $x \rightarrow z \rightarrow u$ ?

Интуиция: что делают шаги  $x \rightarrow z \rightarrow u$ ?

**Шаг**  $x$  — реши свою половину, держа  $z$   
фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$



## Интуиция: что делают шаги $x \rightarrow z \rightarrow u$ ?

**Шаг**  $x$  — реши свою половину, держа  $z$  фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

**Шаг**  $z$  — подстройся под новый  $x$ :

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

## Интуиция: что делают шаги $x \rightarrow z \rightarrow u$ ?

**Шаг  $x$**  — реши свою половину, держа  $z$  фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

**Шаг  $z$**  — подстройся под новый  $x$ :

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

**Шаг  $u$**  — накопи рассогласование (интегральная обратная связь):

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

Здесь  $u$  — масштабированный множитель ( $\nu = \rho u$ ), поэтому  $\rho$  ушёл внутрь нормы.

## Интуиция: что делают шаги $x \rightarrow z \rightarrow u$ ?

**Шаг  $x$**  — реши свою половину, держа  $z$  фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

**Шаг  $z$**  — подстройся под новый  $x$ :

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

**Шаг  $u$**  — накопи рассогласование (интегральная обратная связь):

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

Здесь  $u$  — масштабированный множитель ( $\nu = \rho u$ ), поэтому  $\rho$  ушёл внутрь нормы.

**Интерпретация.** Шаги  $x$  и  $z$  тянут общее решение каждый к своему оптимуму ( $x$  отвечает за гладкую часть,  $z$  — за негладкую часть и ограничение). Множитель  $u$  растёт ровно настолько, насколько расходятся  $x$  и  $z$ , и штрафом сближает их.

## Интуиция: что делают шаги $x \rightarrow z \rightarrow u$ ?

**Шаг  $x$**  — реши свою половину, держа  $z$  фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

**Шаг  $z$**  — подстройся под новый  $x$ :

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

**Шаг  $u$**  — накопи рассогласование (интегральная обратная связь):

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

Здесь  $u$  — масштабированный множитель ( $\nu = \rho u$ ), поэтому  $\rho$  ушёл внутрь нормы.

**Интерпретация.** Шаги  $x$  и  $z$  тянут общее решение каждый к своему оптимуму ( $x$  отвечает за гладкую часть,  $z$  — за негладкую часть и ограничение). Множитель  $u$  растёт ровно настолько, насколько расходятся  $x$  и  $z$ , и штрафом сближает их.

- $\rho$  играет роль жёсткости связи между  $x$  и  $z$ : при большом  $\rho$  согласование быстрее, но оптимизация каждой части медленнее.

# ADMM и проксимальный метод: одна задача, две записи

Композитная задача с гладким  $f$  и негладким  $r$  (как в лекции 14):

$$\min_x f(x) + r(x).$$

## Форма ADMM

Расщепляем переменную ( $A = I$ ,  $B = -I$ ,  $c = 0$ ):

$\min_{x,z} f(x) + r(z)$  при  $x = z$ . Масштабированная форма:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|x - z_{k-1} + u_{k-1}\|^2$$

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|x_k - z + u_{k-1}\|^2$$

$$= \text{prox}_{r, 1/\rho}(x_k + u_{k-1})$$

$$u_k = u_{k-1} + (x_k - z_k)$$

Шаг  $z$  — это в точности проксимальный оператор из лекции 14 (при  $\alpha = 1/\rho$ ).

# ADMM и проксимальный метод: одна задача, две записи

Композитная задача с гладким  $f$  и негладким  $r$  (как в лекции 14):

$$\min_x f(x) + r(x).$$

## Форма ADMM

Расщепляем переменную ( $A = I$ ,  $B = -I$ ,  $c = 0$ ):  
 $\min_{x,z} f(x) + r(z)$  при  $x = z$ . Масштабированная форма:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|x - z_{k-1} + u_{k-1}\|^2$$

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|x_k - z + u_{k-1}\|^2$$

$$= \text{prox}_{r, 1/\rho}(x_k + u_{k-1})$$

$$u_k = u_{k-1} + (x_k - z_k)$$

Шаг  $z$  — это в точности проксимальный оператор из лекции 14 (при  $\alpha = 1/\rho$ ).

## Проксимальный градиентный метод

Та же задача без расщепления: гладкую часть берём градиентным шагом, негладкую — проксом (лекция 14):

$$x_{k+1} = \text{prox}_{r, \alpha}(x_k - \alpha \nabla f(x_k)),$$

$$\text{prox}_{r, \alpha}(y) = \arg \min_x \left[ r(x) + \frac{1}{2\alpha} \|x - y\|^2 \right].$$

Один прокс негладкой части  $r$  за итерацию; для шага по  $f$  нужен градиент  $\nabla f$ .

## Невязки ADMM: критерий остановки и настройка $\rho$

## Невязки ADMM: критерий остановки и настройка $\rho$

**Прямая невязка** измеряет нарушение ограничения (рассогласование  $x$  и  $z$ ):

$$r_k = Ax_k + Bz_k - c$$



## Невязки ADMM: критерий остановки и настройка $\rho$

**Прямая невязка** измеряет нарушение ограничения (рассогласование  $x$  и  $z$ ):

$$r_k = Ax_k + Bz_k - c$$

**Двойственная невязка** измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

## Невязки ADMM: критерий остановки и настройка $\rho$

**Прямая невязка** измеряет нарушение ограничения (рассогласование  $x$  и  $z$ ):

$$r_k = Ax_k + Bz_k - c$$

**Двойственная невязка** измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

Откуда: оптимальность шага  $x$  даёт

$0 \in \partial f(x_k) + A^T(\nu_{k-1} + \rho(Ax_k + Bz_{k-1} - c))$ . Подставив

$\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$ , получаем

$s_k = \rho A^T B(z_k - z_{k-1}) \in \partial f(x_k) + A^T \nu_k$ . То есть  $s_k$  показывает, насколько нарушено условие двойственной допустимости

$0 \in \partial f(x_k) + A^T \nu_k$ ; как только  $z$  перестаёт меняться ( $z_k = z_{k-1}$ ), невязка  $s_k$  обращается в ноль.

## Невязки ADMM: критерий остановки и настройка $\rho$

**Прямая невязка** измеряет нарушение ограничения (рассогласование  $x$  и  $z$ ):

$$r_k = Ax_k + Bz_k - c$$

**Двойственная невязка** измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

Откуда: оптимальность шага  $x$  даёт

$0 \in \partial f(x_k) + A^T(\nu_{k-1} + \rho(Ax_k + Bz_{k-1} - c))$ . Подставив

$\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$ , получаем

$s_k = \rho A^T B(z_k - z_{k-1}) \in \partial f(x_k) + A^T \nu_k$ . То есть  $s_k$  показывает, насколько нарушено условие двойственной допустимости

$0 \in \partial f(x_k) + A^T \nu_k$ ; как только  $z$  перестаёт меняться ( $z_k = z_{k-1}$ ), невязка  $s_k$  обращается в ноль.

В ККТ-оптимуме **оба равны нулю**. Критерий остановки:

$$\|r_k\| \leq \varepsilon^{\text{pri}}, \quad \|s_k\| \leq \varepsilon^{\text{dual}}.$$

## Невязки ADMM: критерий остановки и настройка $\rho$

**Прямая невязка** измеряет нарушение ограничения (рассогласование  $x$  и  $z$ ):

$$r_k = Ax_k + Bz_k - c$$

**Двойственная невязка** измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

Откуда: оптимальность шага  $x$  даёт

$0 \in \partial f(x_k) + A^T(\nu_{k-1} + \rho(Ax_k + Bz_{k-1} - c))$ . Подставив

$\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$ , получаем

$s_k = \rho A^T B(z_k - z_{k-1}) \in \partial f(x_k) + A^T \nu_k$ . То есть  $s_k$  показывает, насколько нарушено условие двойственной допустимости

$0 \in \partial f(x_k) + A^T \nu_k$ ; как только  $z$  перестаёт меняться ( $z_k = z_{k-1}$ ),

невязка  $s_k$  обращается в ноль.

В ККТ-оптимуме **оба равны нулю**. Критерий остановки:

$$\|r_k\| \leq \varepsilon^{\text{pri}}, \quad \|s_k\| \leq \varepsilon^{\text{dual}}.$$

**Баланс  $\rho$ :**

- $\|r_k\| \gg \|s_k\|$ : ограничение нарушено сильнее  $\Rightarrow$  **увеличить  $\rho$** .

## Невязки ADMM: критерий остановки и настройка $\rho$

**Прямая невязка** измеряет нарушение ограничения (рассогласование  $x$  и  $z$ ):

$$r_k = Ax_k + Bz_k - c$$

**Двойственная невязка** измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

Откуда: оптимальность шага  $x$  даёт

$0 \in \partial f(x_k) + A^T(\nu_{k-1} + \rho(Ax_k + Bz_{k-1} - c))$ . Подставив

$\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$ , получаем

$s_k = \rho A^T B(z_k - z_{k-1}) \in \partial f(x_k) + A^T \nu_k$ . То есть  $s_k$  показывает, насколько нарушено условие двойственной допустимости

$0 \in \partial f(x_k) + A^T \nu_k$ ; как только  $z$  перестаёт меняться ( $z_k = z_{k-1}$ ),

невязка  $s_k$  обращается в ноль.

В ККТ-оптимуме **оба равны нулю**. Критерий остановки:

$$\|r_k\| \leq \varepsilon^{\text{pri}}, \quad \|s_k\| \leq \varepsilon^{\text{dual}}.$$

**Баланс  $\rho$ :**

- $\|r_k\| \gg \|s_k\|$ : ограничение нарушено сильнее  $\Rightarrow$  **увеличить  $\rho$** .
- $\|s_k\| \gg \|r_k\|$ : решение слишком инертно  $\Rightarrow$  **уменьшить  $\rho$** .

## Невязки ADMM: критерий остановки и настройка $\rho$

**Прямая невязка** измеряет нарушение ограничения (рассогласование  $x$  и  $z$ ):

$$r_k = Ax_k + Bz_k - c$$

**Двойственная невязка** измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

Откуда: оптимальность шага  $x$  даёт

$0 \in \partial f(x_k) + A^T(\nu_{k-1} + \rho(Ax_k + Bz_{k-1} - c))$ . Подставив

$\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$ , получаем

$s_k = \rho A^T B(z_k - z_{k-1}) \in \partial f(x_k) + A^T \nu_k$ . То есть  $s_k$  показывает, насколько нарушено условие двойственной допустимости

$0 \in \partial f(x_k) + A^T \nu_k$ ; как только  $z$  перестаёт меняться ( $z_k = z_{k-1}$ ),

невязка  $s_k$  обращается в ноль.

В ККТ-оптимуме **оба равны нулю**. Критерий остановки:

$$\|r_k\| \leq \varepsilon^{\text{pri}}, \quad \|s_k\| \leq \varepsilon^{\text{dual}}.$$

**Баланс  $\rho$ :**

- $\|r_k\| \gg \|s_k\|$ : ограничение нарушено сильнее  $\Rightarrow$  **увеличить  $\rho$** .
- $\|s_k\| \gg \|r_k\|$ : решение слишком инертно  $\Rightarrow$  **уменьшить  $\rho$** .

## Невязки ADMM: критерий остановки и настройка $\rho$

**Прямая невязка** измеряет нарушение ограничения (рассогласование  $x$  и  $z$ ):

$$r_k = Ax_k + Bz_k - c$$

**Двойственная невязка** измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

Откуда: оптимальность шага  $x$  даёт

$0 \in \partial f(x_k) + A^T(\nu_{k-1} + \rho(Ax_k + Bz_{k-1} - c))$ . Подставив

$\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$ , получаем

$s_k = \rho A^T B(z_k - z_{k-1}) \in \partial f(x_k) + A^T \nu_k$ . То есть  $s_k$  показывает, насколько нарушено условие двойственной допустимости

$0 \in \partial f(x_k) + A^T \nu_k$ ; как только  $z$  перестаёт меняться ( $z_k = z_{k-1}$ ),

невязка  $s_k$  обращается в ноль.

В ККТ-оптимуме **оба равны нулю**. Критерий остановки:

$$\|r_k\| \leq \varepsilon^{\text{pri}}, \quad \|s_k\| \leq \varepsilon^{\text{dual}}.$$

**Баланс  $\rho$ :**

- $\|r_k\| \gg \|s_k\|$ : ограничение нарушено сильнее  $\Rightarrow$  **увеличить  $\rho$** .
- $\|s_k\| \gg \|r_k\|$ : решение слишком инертно  $\Rightarrow$  **уменьшить  $\rho$** .

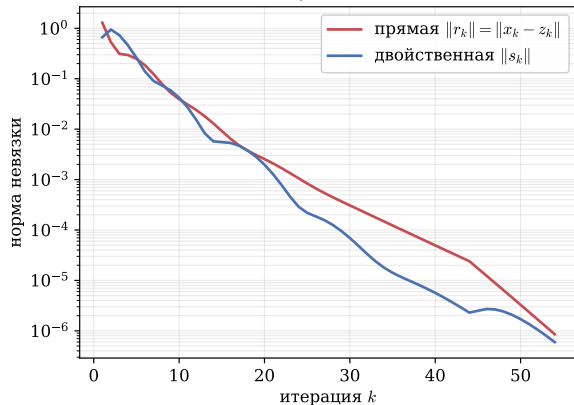
**Адаптивное правило** (Boyd et al., §3.4.1):

$$\rho_{k+1} = \begin{cases} 2\rho_k, & \|r_k\| > 10\|s_k\| \\ \rho_k/2, & \|s_k\| > 10\|r_k\| \\ \rho_k, & \text{иначе} \end{cases}$$

Простая эвристика, которая часто заметно ускоряет сходимость.

# Невязки и адаптивный $\rho$ : эксперимент

Адаптивный  $\rho$ : обе невязки  $\rightarrow 0$



Цена неудачного  $\rho$  против адаптивного

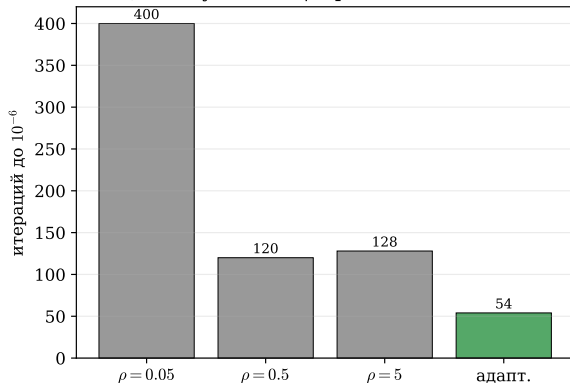


Figure 1: Слева: прямая и двойственная невязки ADMM на LASSO с адаптивным  $\rho$ , обе монотонно падают до  $10^{-6}$ . Справа: число итераций до этой точности; неудачно выбранное фиксированное  $\rho = 0.05$  заметно медленнее, а правило балансировки невязок сходится быстрее всех протестированных фиксированных  $\rho$  без ручного подбора (с одной рефакторизацией).



## Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое  $f_i$  хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

## Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое  $f_i$  хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии**  $x_i$  и общую консенсус-переменную  $z$ :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

## Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое  $f_i$  хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии**  $x_i$  и общую консенсус-переменную  $z$ :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

Итерации ADMM становятся **полностью параллельными**:

$$x_i^k = \arg \min_{x_i} f_i(x_i) + \frac{\rho}{2} \|x_i - z^{k-1} + u_i^{k-1}\|^2$$

$$z^k = \frac{1}{N} \sum_{i=1}^N (x_i^k + u_i^{k-1})$$

$$u_i^k = u_i^{k-1} + x_i^k - z^k$$

## Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое  $f_i$  хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии**  $x_i$  и общую консенсус-переменную  $z$ :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

Итерации ADMM становятся **полностью параллельными**:

$$x_i^k = \arg \min_{x_i} f_i(x_i) + \frac{\rho}{2} \|x_i - z^{k-1} + u_i^{k-1}\|^2$$

$$z^k = \frac{1}{N} \sum_{i=1}^N (x_i^k + u_i^{k-1})$$

$$u_i^k = u_i^{k-1} + x_i^k - z^k$$

**Структура связи: локально  $\rightarrow$  глобально**

- **Локально (параллельно):** каждая машина решает свою задачу  $x_i$  по своим данным.

При  $\sum_i u_i^0 = 0$  имеем  $z^k = \frac{1}{N} \sum_i x_i^k$ .

# Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое  $f_i$  хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии**  $x_i$  и общую консенсус-переменную  $z$ :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

Итерации ADMM становятся **полностью параллельными**:

$$x_i^k = \arg \min_{x_i} f_i(x_i) + \frac{\rho}{2} \|x_i - z^{k-1} + u_i^{k-1}\|^2$$

$$z^k = \frac{1}{N} \sum_{i=1}^N (x_i^k + u_i^{k-1})$$

$$u_i^k = u_i^{k-1} + x_i^k - z^k$$

**Структура связи: локально  $\rightarrow$  глобально**

- **Локально (параллельно):** каждая машина решает свою задачу  $x_i$  по своим данным.
- **Глобально (усреднение):**  $z$  есть **усреднение** локальных решений (и их множителей).

При  $\sum_i u_i^0 = 0$  имеем  $z^k = \frac{1}{N} \sum_i x_i^k$ .

# Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое  $f_i$  хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии**  $x_i$  и общую консенсус-переменную  $z$ :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

Итерации ADMM становятся **полностью параллельными**:

$$x_i^k = \arg \min_{x_i} f_i(x_i) + \frac{\rho}{2} \|x_i - z^{k-1} + u_i^{k-1}\|^2$$

$$z^k = \frac{1}{N} \sum_{i=1}^N (x_i^k + u_i^{k-1})$$

$$u_i^k = u_i^{k-1} + x_i^k - z^k$$

**Структура связи: локально  $\rightarrow$  глобально**

- **Локально (параллельно):** каждая машина решает свою задачу  $x_i$  по своим данным.
- **Глобально (усреднение):**  $z$  есть **усреднение** локальных решений (и их множителей).

При  $\sum_i u_i^0 = 0$  имеем  $z^k = \frac{1}{N} \sum_i x_i^k$ .

## Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое  $f_i$  хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии**  $x_i$  и общую консенсус-переменную  $z$ :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

Итерации ADMM становятся **полностью параллельными**:

$$x_i^k = \arg \min_{x_i} f_i(x_i) + \frac{\rho}{2} \|x_i - z^{k-1} + u_i^{k-1}\|^2$$

$$z^k = \frac{1}{N} \sum_{i=1}^N (x_i^k + u_i^{k-1})$$

$$u_i^k = u_i^{k-1} + x_i^k - z^k$$

**Структура связи: локально  $\rightarrow$  глобально**

- **Локально (параллельно):** каждая машина решает свою задачу  $x_i$  по своим данным.
- **Глобально (усреднение):**  $z$  есть **усреднение** локальных решений (и их множителей).

При  $\sum_i u_i^0 = 0$  имеем  $z^k = \frac{1}{N} \sum_i x_i^k$ .

Так ADMM применяют в федеративном обучении: данные не покидают узел, обмен идёт только векторами.

# Консенсусный ADMM в действии

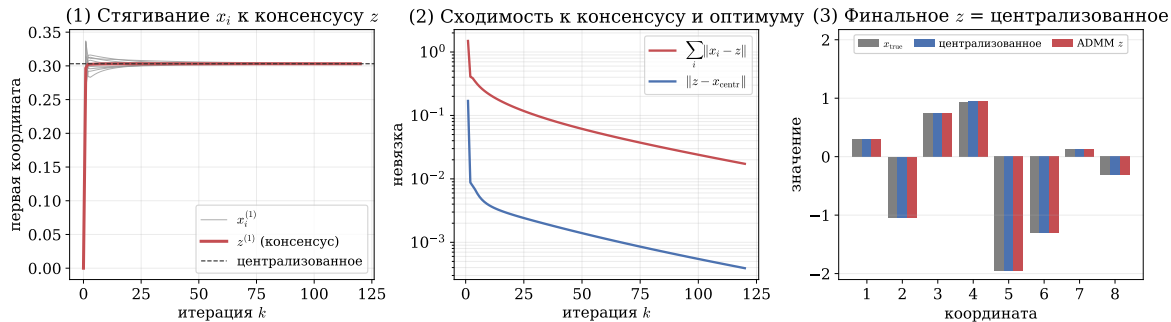


Figure 2: Распределённый МНК,  $N = 10$  узлов. Слева: первые координаты локальных  $x_i$  (серые) стягиваются к консенсусу  $z$  (красная) и к централизованному решению. В центре: расхождение по координатам  $\sum_i \|x_i - z\|$  и отклонение  $\|z - x_{\text{centr}}\|$  от централизованного решения затухают. Справа: итоговое  $z$  совпадает с централизованным; по сети ходят только векторы.



## Применения ADMM: один общий приём

За всеми применениями стоит один приём: негладкую часть  $r$  отдаём  $z$ -шагу, где  $\text{prox}_r$  имеет **замкнутую форму**; гладкую часть оставляем  $x$ -шагу.

## Применения ADMM: один общий приём

За всеми применениями стоит один приём: негладкую часть  $r$  отдаём  $z$ -шагу, где  $\text{prox}_r$  имеет **замкнутую форму**; гладкую часть оставляем  $x$ -шагу.

| $r(z)$            | $z$ -шаг ( $\text{prox}_r$ ) | где применяется               |
|-------------------|------------------------------|-------------------------------|
| $\lambda \ z\ _1$ | мягкий порог                 | LASSO, разреженная регрессия  |
| TV-штраф          | покоординатное сжатие        | TV-восстановление изображений |
| $\ Z\ _*$         | усечение сингулярных чисел   | robust PCA (низкий ранг)      |
| $I_{\mathcal{C}}$ | проекция на $\mathcal{C}$    | конусные ограничения          |

## Применения ADMM: где он нужен

- Разреженная регрессия:  $\ell_1$ , group LASSO.

## Применения ADMM: где он нужен

- Разреженная регрессия:  $\ell_1$ , group LASSO.
- TV-обработка изображений: точная негладкая задача (см. далее).

## Применения ADMM: где он нужен

- Разреженная регрессия:  $\ell_1$ , group LASSO.
- TV-обработка изображений: точная негладкая задача (см. далее).
- Распределённое и федеративное обучение: консенсусный ADMM, данные остаются на узлах.

## Применения ADMM: где он нужен

- Разреженная регрессия:  $\ell_1$ , group LASSO.
- TV-обработка изображений: точная негладкая задача (см. далее).
- Распределённое и федеративное обучение: консенсусный ADMM, данные остаются на узлах.
- Низкий ранг плюс разреженность (robust PCA): методы первого порядка здесь не разбивают задачу.

## Применения ADMM: где он нужен

- Разреженная регрессия:  $\ell_1$ , group LASSO.
- TV-обработка изображений: точная негладкая задача (см. далее).
- Распределённое и федеративное обучение: консенсусный ADMM, данные остаются на узлах.
- Низкий ранг плюс разреженность (robust PCA): методы первого порядка здесь не разбивают задачу.

## Применения ADMM: где он нужен

- Разреженная регрессия:  $\ell_1$ , group LASSO.
- TV-обработка изображений: точная негладкая задача (см. далее).
- Распределённое и федеративное обучение: консенсусный ADMM, данные остаются на узлах.
- Низкий ранг плюс разреженность (robust PCA): методы первого порядка здесь не разбивают задачу.

**Оговорка.** На простом LASSO к высокой точности FISTA сопоставим или быстрее. ADMM выигрывает там, где есть несколько негладких частей, оператор внутри регуляризатора, жёсткие ограничения или распределённость (fused LASSO, TV, консенсус, robust PCA).



# ADMM на практике: восстановление смазанного изображения

Смазано + шум (25.9 дБ)



ADMM (28.5 дБ)

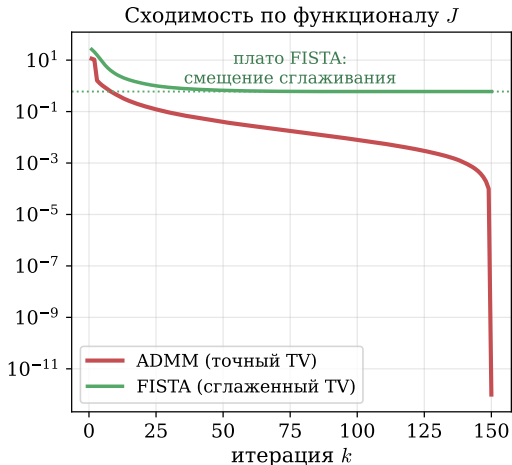


Figure 3: Устранение размытия настоящего фото. ADMM решает **точную** негладкую TV-задачу (FISTA минимизирует **сглаженный** суррогат и смещён по  $J$ ). По качеству (PSNR) методы сопоставимы: FISTA 28.7 дБ, ADMM 28.5 дБ; ADMM ценен структурой, а не скоростью.

# ADMM на практике: восстановление смазанного изображения

Смазано + шум (25.9 дБ)



ADMM (28.5 дБ)

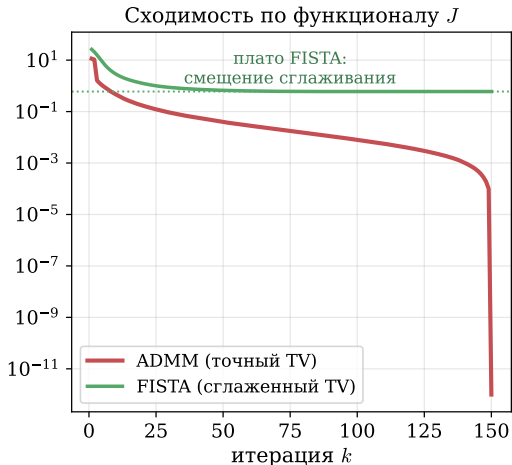


Figure 3: Устранение размытия настоящего фото. ADMM решает **точную** негладкую TV-задачу (FISTA минимизирует **сглаженный** суррогат и смещён по  $J$ ). По качеству (PSNR) методы сопоставимы: FISTA 28.7 дБ, ADMM 28.5 дБ; ADMM ценен структурой, а не скоростью.

# ADMM как метод по умолчанию: robust PCA

Robust PCA через ADMM:  $\min \|L\|_* + \lambda \|S\|_1$  при  $L + S = M$

Кадр  $M$  (фон+объект)



Низкоранг  $L$  (фон)



Разреж  $|S|$  (движение)



Figure 4: Вычитание фона: кадры видео суть столбцы матрицы  $M$ . ADMM решает  $\min \|L\|_* + \lambda \|S\|_1$  при  $L + S = M$ :  $L$  (низкий ранг) — статичный фон,  $S$  (разреженная компонента) — движущийся объект.

Шаги ADMM в замкнутой форме:

- $L$ -шаг: пороговая обработка сингулярных чисел (прокс ядерной нормы);

# ADMM как метод по умолчанию: robust PCA

Robust PCA через ADMM:  $\min \|L\|_* + \lambda \|S\|_1$  при  $L + S = M$

Кадр  $M$  (фон+объект)



Низкоранг  $L$  (фон)



Разреж  $|S|$  (движение)



Шаги ADMM в замкнутой форме:

- $L$ -шаг: пороговая обработка сингулярных чисел (прокс ядерной нормы);
- $S$ -шаг: мягкий порог (прокс  $\ell_1$ );

Figure 4: Вычитание фона: кадры видео суть столбцы матрицы  $M$ . ADMM решает  $\min \|L\|_* + \lambda \|S\|_1$  при  $L + S = M$ :  $L$  (низкий ранг) — статичный фон,  $S$  (разреженная компонента) — движущийся объект.

# ADMM как метод по умолчанию: robust PCA

Robust PCA через ADMM:  $\min \|L\|_* + \lambda \|S\|_1$  при  $L + S = M$

Кадр  $M$  (фон+объект)



Низкоранг  $L$  (фон)



Разреж  $|S|$  (движение)



Figure 4: Вычитание фона: кадры видео суть столбцы матрицы  $M$ . ADMM решает  $\min \|L\|_* + \lambda \|S\|_1$  при  $L + S = M$ :  $L$  (низкий ранг) — статичный фон,  $S$  (разреженная компонента) — движущийся объект.

Шаги ADMM в замкнутой форме:

- $L$ -шаг: пороговая обработка сингулярных чисел (прокс ядерной нормы);
- $S$ -шаг: мягкий порог (прокс  $\ell_1$ );
- $u$ -шаг: обновление множителя.

# ADMM как метод по умолчанию: robust PCA

Robust PCA через ADMM:  $\min \|L\|_* + \lambda \|S\|_1$  при  $L + S = M$

Кадр  $M$  (фон+объект)



Низкоранг  $L$  (фон)



Разреж  $|S|$  (движение)



Figure 4: Вычитание фона: кадры видео суть столбцы матрицы  $M$ . ADMM решает  $\min \|L\|_* + \lambda \|S\|_1$  при  $L + S = M$ :  $L$  (низкий ранг) — статичный фон,  $S$  (разреженная компонента) — движущийся объект.

Шаги ADMM в замкнутой форме:

- $L$ -шаг: пороговая обработка сингулярных чисел (прокс ядерной нормы);
- $S$ -шаг: мягкий порог (прокс  $\ell_1$ );
- $u$ -шаг: обновление множителя.

# ADMM как метод по умолчанию: robust PCA

Robust PCA через ADMM:  $\min \|L\|_* + \lambda \|S\|_1$  при  $L + S = M$

Кадр  $M$  (фон+объект)



Низкоранг  $L$  (фон)



Разреж  $|S|$  (движение)



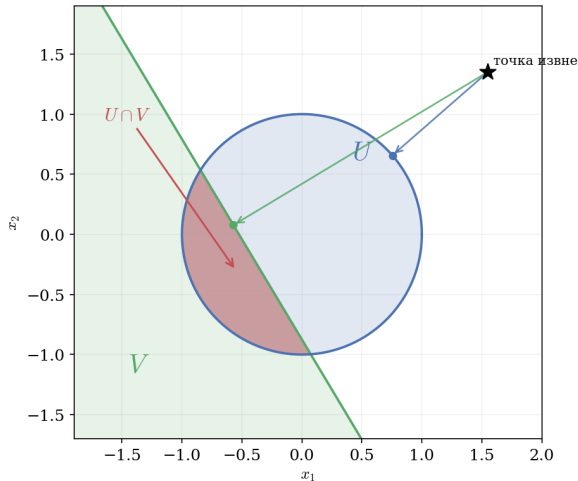
Шаги ADMM в замкнутой форме:

- $L$ -шаг: пороговая обработка сингулярных чисел (прокс ядерной нормы);
- $S$ -шаг: мягкий порог (прокс  $\ell_1$ );
- $u$ -шаг: обновление множителя.

Методы первого порядка не разбивают «ядерная норма +  $\ell_1$ » так естественно; здесь ADMM есть стандарт.

Figure 4: Вычитание фона: кадры видео суть столбцы матрицы  $M$ . ADMM решает  $\min \|L\|_* + \lambda \|S\|_1$  при  $L + S = M$ :  $L$  (низкий ранг) — статичный фон,  $S$  (разреженная компонента) — движущийся объект.

## Пример: метод альтернирующих проекций



Поиск точки в пересечении выпуклых множеств  $U, V \subseteq \mathbb{R}^n$ :

$$\min_x I_U(x) + I_V(x)$$

Приводим к форме ADMM ( $A = I, B = -I, c = 0$ ):

$$\min_{x,z} I_U(x) + I_V(z) \quad \text{при} \quad x - z = 0$$

Каждый цикл ADMM включает две проекции:

$$x_k = P_U(z_{k-1} - u_{k-1})$$

$$z_k = P_V(x_k + u_{k-1})$$

$$u_k = u_{k-1} + x_k - z_k$$

Коррекция  $u$  даёт сходимость там, где простые поочерёдные проекции зацикливаются.

Figure 5: Сложную проекцию на пересечение  $U \cap V$  (красный «серп») ADMM заменяет двумя простыми: на круг  $U$  и на полуплоскость  $V$  по отдельности.



## Проекция на пересечение: ADMM в действии

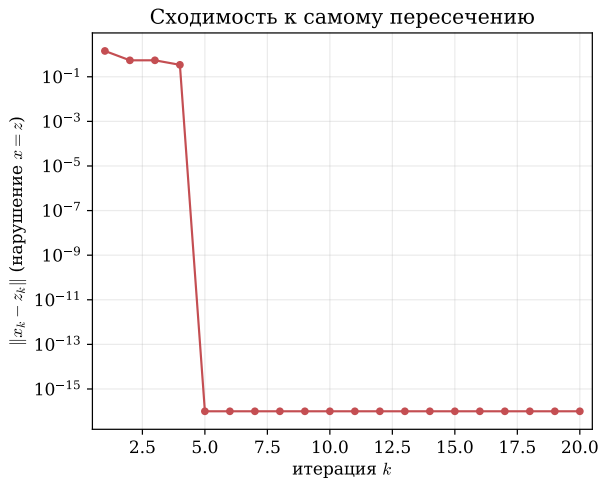
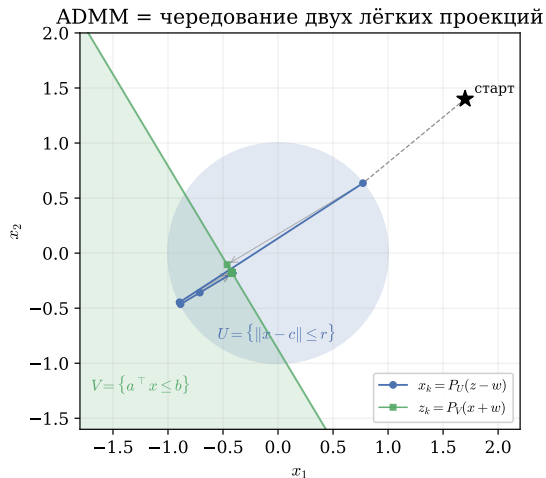


Figure 6: ADMM как чередующиеся проекции на круг  $U$  и полуплоскость  $V$ . Траектория зигзагом сходится в пересечение,  $\|x_k - z_k\| \rightarrow 0$ : сложную проекцию заменяем двумя простыми.

## LASSO через ADMM: расщепить гладкое и негладкое

LASSO  $\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1$ : два слагаемых **разной природы**. Вводим копию  $z = x$  и отдаём каждое слагаемое своему методу:

$$\min_{x,z} \frac{1}{2} \|Ax - b\|^2 + \lambda \|z\|_1 \quad \text{при } x = z$$

## LASSO через ADMM: расщепить гладкое и негладкое

LASSO  $\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1$ : два слагаемых **разной природы**. Вводим копию  $z = x$  и отдаём каждое слагаемое своему методу:

$$\min_{x,z} \frac{1}{2} \|Ax - b\|^2 + \lambda \|z\|_1 \quad \text{при } x = z$$

Шаги ADMM (масштабированная форма  $u$ ):

$$x_{k+1} = (A^T A + \rho I)^{-1} (A^T b + \rho(z_k - u_k)),$$

$$z_{k+1} = S_{\lambda/\rho}(x_{k+1} + u_k),$$

$$u_{k+1} = u_k + (x_{k+1} - z_{k+1}),$$

где  $S_\tau(v) = \text{sign}(v) \cdot \max(|v| - \tau, 0)$  — мягкий порог.

|            | $\frac{1}{2} \ Ax - b\ ^2$ | $\lambda \ z\ _1$ |
|------------|----------------------------|-------------------|
| гладкость  | да                         | нет               |
| инструмент | система                    | мягкий порог      |
| сложность  | матрица $A$                | покоординатно     |

# LASSO через ADMM: расщепить гладкое и негладкое

LASSO  $\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1$ : два слагаемых **разной природы**. Вводим копию  $z = x$  и отдаём каждое слагаемое своему методу:

$$\min_{x,z} \frac{1}{2} \|Ax - b\|^2 + \lambda \|z\|_1 \quad \text{при } x = z$$

Шаги ADMM (масштабированная форма  $u$ ):

|            | $\frac{1}{2} \ Ax - b\ ^2$ | $\lambda \ z\ _1$ |
|------------|----------------------------|-------------------|
| гладкость  | да                         | нет               |
| инструмент | система                    | мягкий порог      |
| сложность  | матрица $A$                | покоординатно     |

$$x_{k+1} = (A^T A + \rho I)^{-1} (A^T b + \rho (z_k - u_k)),$$

$$z_{k+1} = S_{\lambda/\rho}(x_{k+1} + u_k),$$

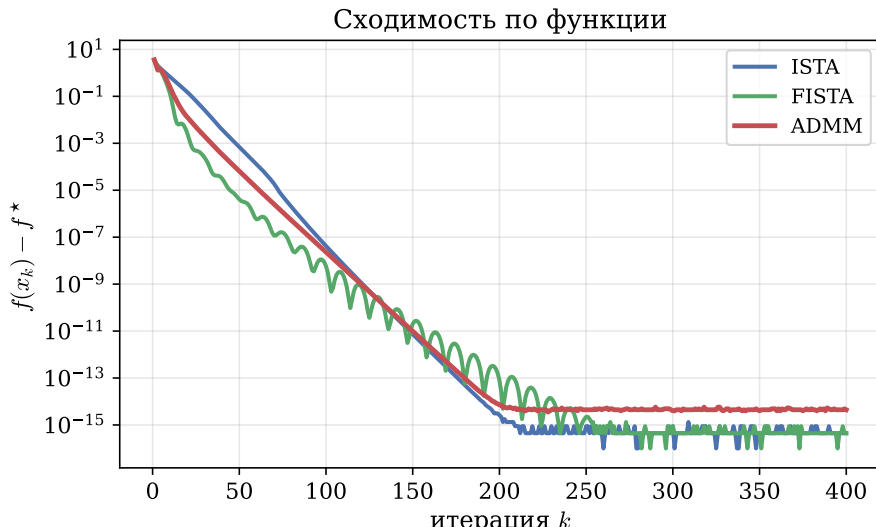
$$u_{k+1} = u_k + (x_{k+1} - z_{k+1}),$$

где  $S_\tau(v) = \text{sign}(v) \cdot \max(|v| - \tau, 0)$  — мягкий порог.

$x$ -шаг есть линейная система (естественная для квадратичной части),  $z$ -шаг есть мягкий порог в замкнутой форме (естественный для  $\ell_1$ ),  $u$ -шаг есть цена за рассогласование  $x \neq z$ . Факторизация  $(A^T A + \rho I)$  считается один раз.

# LASSO через ADMM: сравнение с ISTA / FISTA

LASSO:  $m = 200$ ,  $n = 500$ ,  $\|x^*\|_0 = 25$



## Где ADMM явно выигрывает: оператор в регуляризаторе

Возьмём **fused LASSO**  $\min_x \frac{1}{2} \|x - y\|^2 + \lambda \|Dx\|_1$ , где  $D$  — разностная матрица,  $(Dx)_i = x_{i+1} - x_i$ . У  $\lambda \|Dx\|_1$  нет замкнутого прох (разность связывает координаты), поэтому прокс-градиент напрямую неприменим; прямой конкурент — субградиентный метод.

## Где ADMM явно выигрывает: оператор в регуляризаторе

Возьмём **fused LASSO**  $\min_x \frac{1}{2} \|x - y\|^2 + \lambda \|Dx\|_1$ , где  $D$  — разностная матрица,  $(Dx)_i = x_{i+1} - x_i$ . У  $\lambda \|Dx\|_1$  нет замкнутого прох (разность связывает координаты), поэтому прокс-градиент напрямую неприменим; прямой конкурент — субградиентный метод.

**Приём ADMM.** Вытащим оператор  $D$  из-под нормы в линейное ограничение, введя  $z = Dx$  ( $A = D$ ,  $B = -I$ ,  $c = 0$ ):

$$\min_{x,z} \frac{1}{2} \|x - y\|^2 + \lambda \|z\|_1 \quad \text{при} \quad Dx - z = 0.$$

## Где ADMM явно выигрывает: оператор в регуляризаторе

Возьмём **fused LASSO**  $\min_x \frac{1}{2} \|x - y\|^2 + \lambda \|Dx\|_1$ , где  $D$  — разностная матрица,  $(Dx)_i = x_{i+1} - x_i$ . У  $\lambda \|Dx\|_1$  нет замкнутого прох (разность связывает координаты), поэтому прокс-градиент напрямую неприменим; прямой конкурент — субградиентный метод.

**Приём ADMM.** Вытащим оператор  $D$  из-под нормы в линейное ограничение, введя  $z = Dx$  ( $A = D$ ,  $B = -I$ ,  $c = 0$ ):

$$\min_{x,z} \frac{1}{2} \|x - y\|^2 + \lambda \|z\|_1 \quad \text{при} \quad Dx - z = 0.$$

**Шаг  $x$  — гладкий (квадратичный).** Линейная система

$$(I + \rho D^T D) x_k = y + \rho D^T (z_{k-1} - u_{k-1}).$$

$D^T D$  трёхдиагональна  $\Rightarrow$  решение за  $O(n)$  (одна факторизация на весь запуск).



## Где ADMM явно выигрывает: оператор в регуляризаторе

Возьмём **fused LASSO**  $\min_x \frac{1}{2} \|x - y\|^2 + \lambda \|Dx\|_1$ , где  $D$  — разностная матрица,  $(Dx)_i = x_{i+1} - x_i$ . У  $\lambda \|Dx\|_1$  нет замкнутого прох (разность связывает координаты), поэтому прокс-градиент напрямую неприменим; прямой конкурент — субградиентный метод.

**Приём ADMM.** Вытащим оператор  $D$  из-под нормы в линейное ограничение, введя  $z = Dx$  ( $A = D$ ,  $B = -I$ ,  $c = 0$ ):

$$\min_{x,z} \frac{1}{2} \|x - y\|^2 + \lambda \|z\|_1 \quad \text{при} \quad Dx - z = 0.$$

**Шаг  $x$  — гладкий (квадратичный).** Линейная система

$$(I + \rho D^T D) x_k = y + \rho D^T (z_{k-1} - u_{k-1}).$$

$D^T D$  трёхдиагональна  $\Rightarrow$  решение за  $O(n)$  (одна факторизация на весь запуск).

**Шаг  $z$  — негладкий, но уже без  $D$ .**

$$z_k = \text{prox}_{\lambda \|\cdot\|_1, 1/\rho} (Dx_k + u_{k-1}),$$

покоординатный мягкий порог в замкнутой форме. И  $u_k = u_{k-1} + (Dx_k - z_k)$ .

## Где ADMM явно выигрывает: оператор в регуляризаторе

Возьмём **fused LASSO**  $\min_x \frac{1}{2} \|x - y\|^2 + \lambda \|Dx\|_1$ , где  $D$  — разностная матрица,  $(Dx)_i = x_{i+1} - x_i$ . У  $\lambda \|Dx\|_1$  нет замкнутого прох (разность связывает координаты), поэтому прокс-градиент напрямую неприменим; прямой конкурент — субградиентный метод.

**Приём ADMM.** Вытащим оператор  $D$  из-под нормы в линейное ограничение, введя  $z = Dx$  ( $A = D$ ,  $B = -I$ ,  $c = 0$ ):

$$\min_{x,z} \frac{1}{2} \|x - y\|^2 + \lambda \|z\|_1 \quad \text{при} \quad Dx - z = 0.$$

**Шаг  $x$  — гладкий (квадратичный).** Линейная система

$$(I + \rho D^T D) x_k = y + \rho D^T (z_{k-1} - u_{k-1}).$$

$D^T D$  трёхдиагональна  $\Rightarrow$  решение за  $O(n)$  (одна факторизация на весь запуск).

**Шаг  $z$  — негладкий, но уже без  $D$ .**

$$z_k = \text{prox}_{\lambda \|\cdot\|_1, 1/\rho} (Dx_k + u_{k-1}),$$

покоординатный мягкий порог в замкнутой форме. И  $u_k = u_{k-1} + (Dx_k - z_k)$ .

ADMM **разносит трудность на две переменные**: оператор  $D$  уходит в гладкий  $x$ -шаг (линейное решение),  $\ell_1$  остаётся на  $z$  с тривиальным проксом. Прокс-градиенту так нельзя — ему нужен прокс от всего  $\lambda \|Dx\|_1$ , а его нет.

## Fused LASSO: численный эксперимент

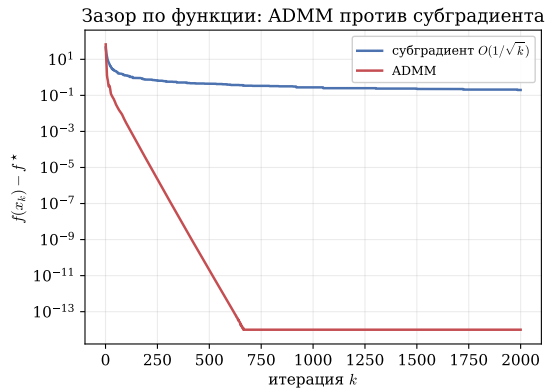
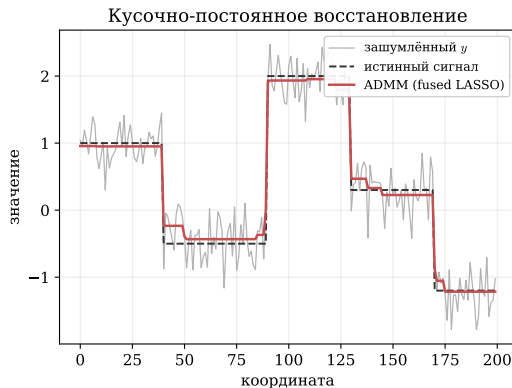


Figure 8: Слева: восстановление кусочно-постоянного сигнала из шума. Справа: ADMM ( $z = Dx$ ) за  $\sim 700$  итераций доходит до машинной точности, субградиент застревает на  $\sim 0.2$ .

# Эволюция методов курса на LASSO

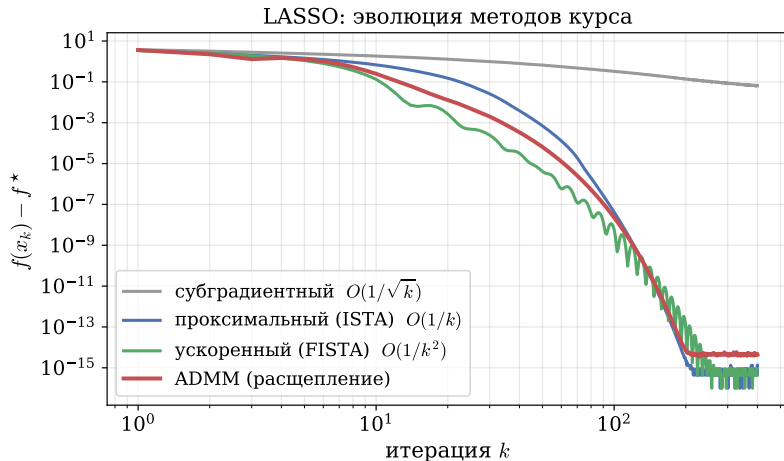


Figure 9: Субградиентный метод сходится медленно ( $O(1/\sqrt{k})$ ) и застревает; проксимальный и ускоренный используют прох негладкой части; ADMM расщепляет задачу. Именно прох и расщепление есть причина перехода от субградиента к этим методам.

## ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

## ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

**OSQP** (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи**

$$\frac{1}{2}x^\top Px + q^\top x \text{ при } l \leq Ax \leq u.$$

## ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

**OSQP** (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи**  
 $\frac{1}{2}x^\top Px + q^\top x$  при  $l \leq Ax \leq u$ .
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.

## ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

**OSQP** (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи**  
 $\frac{1}{2}x^\top Px + q^\top x$  при  $l \leq Ax \leq u$ .
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.
- После настройки работает **без делений**, поддерживает **тёплый старт**, генерирует автономный C-код без динамической памяти.



## ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

**OSQP** (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи**  
 $\frac{1}{2}x^\top Px + q^\top x$  при  $l \leq Ax \leq u$ .
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.
- После настройки работает **без делений**, поддерживает **тёплый старт**, генерирует автономный C-код без динамической памяти.
- Отсюда — рабочая лошадка **встраиваемого MPC** (управление в реальном времени, робототехника) и **QP-движок по умолчанию в CVXPY**.

## ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

**OSQP** (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи**  
 $\frac{1}{2}x^\top Px + q^\top x$  при  $l \leq Ax \leq u$ .
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.
- После настройки работает **без делений**, поддерживает **тёплый старт**, генерирует автономный C-код без динамической памяти.
- Отсюда — рабочая лошадка **встраиваемого MPC** (управление в реальном времени, робототехника) и **QP-движок по умолчанию в CVXPY**.

## ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

**OSQP** (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи**  $\frac{1}{2}x^\top Px + q^\top x$  при  $l \leq Ax \leq u$ .
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.
- После настройки работает **без делений**, поддерживает **тёплый старт**, генерирует автономный C-код без динамической памяти.
- Отсюда — рабочая лошадка **встраиваемого MPC** (управление в реальном времени, робототехника) и **QP-движок по умолчанию в CVXPY**.

**SCS** (*Splitting Conic Solver; O'Donoghue, Chu, Parikh, Boyd, 2016*)

- Решает большие выпуклые **конические задачи**: LP, SOCP, **SDP**, экспоненциальный конус.

## ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

**OSQP** (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи**  $\frac{1}{2}x^\top Px + q^\top x$  при  $l \leq Ax \leq u$ .
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.
- После настройки работает **без делений**, поддерживает **тёплый старт**, генерирует автономный C-код без динамической памяти.
- Отсюда — рабочая лошадка **встраиваемого MPC** (управление в реальном времени, робототехника) и **QP-движок по умолчанию в CVXPY**.

**SCS** (*Splitting Conic Solver; O'Donoghue, Chu, Parikh, Boyd, 2016*)

- Решает большие выпуклые **конические задачи**: LP, SOCP, **SDP**, экспоненциальный конус.
- ADMM поверх однородного самодвойственного вложения; метод первого порядка, масштабируется на большие размерности.

# ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

**OSQP** (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи**  $\frac{1}{2}x^\top Px + q^\top x$  при  $l \leq Ax \leq u$ .
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.
- После настройки работает **без делений**, поддерживает **тёплый старт**, генерирует автономный C-код без динамической памяти.
- Отсюда — рабочая лошадка **встраиваемого MPC** (управление в реальном времени, робототехника) и **QP-движок по умолчанию в CVXPY**.

**SCS** (*Splitting Conic Solver; O'Donoghue, Chu, Parikh, Boyd, 2016*)

- Решает большие выпуклые **конические задачи**: LP, SOCP, **SDP**, экспоненциальный конус.
- ADMM поверх однородного самодвойственного вложения; метод первого порядка, масштабируется на большие размерности.
- Поставляется с CVXPY как универсальный конический солвер.

# ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

**OSQP** (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи**  $\frac{1}{2}x^\top Px + q^\top x$  при  $l \leq Ax \leq u$ .
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.
- После настройки работает **без делений**, поддерживает **тёплый старт**, генерирует автономный C-код без динамической памяти.
- Отсюда — рабочая лошадка **встраиваемого MPC** (управление в реальном времени, робототехника) и **QP-движок по умолчанию в CVXPY**.

**SCS** (*Splitting Conic Solver; O'Donoghue, Chu, Parikh, Boyd, 2016*)

- Решает большие выпуклые **конические задачи**: LP, SOCP, **SDP**, экспоненциальный конус.
- ADMM поверх однородного самодвойственного вложения; метод первого порядка, масштабируется на большие размерности.
- Поставляется с CVXPY как универсальный конический солвер.

# ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

**OSQP** (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи**  $\frac{1}{2}x^\top Px + q^\top x$  при  $l \leq Ax \leq u$ .
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.
- После настройки работает **без делений**, поддерживает **тёплый старт**, генерирует автономный C-код без динамической памяти.
- Отсюда — рабочая лошадка **встраиваемого MPC** (управление в реальном времени, робототехника) и **QP-движок по умолчанию в CVXPY**.

**SCS** (*Splitting Conic Solver; O'Donoghue, Chu, Parikh, Boyd, 2016*)

- Решает большие выпуклые **конические задачи**: LP, SOCP, **SDP**, экспоненциальный конус.
- ADMM поверх однородного самодвойственного вложения; метод первого порядка, масштабируется на большие размерности.
- Поставляется с CVXPY как универсальный конический солвер.

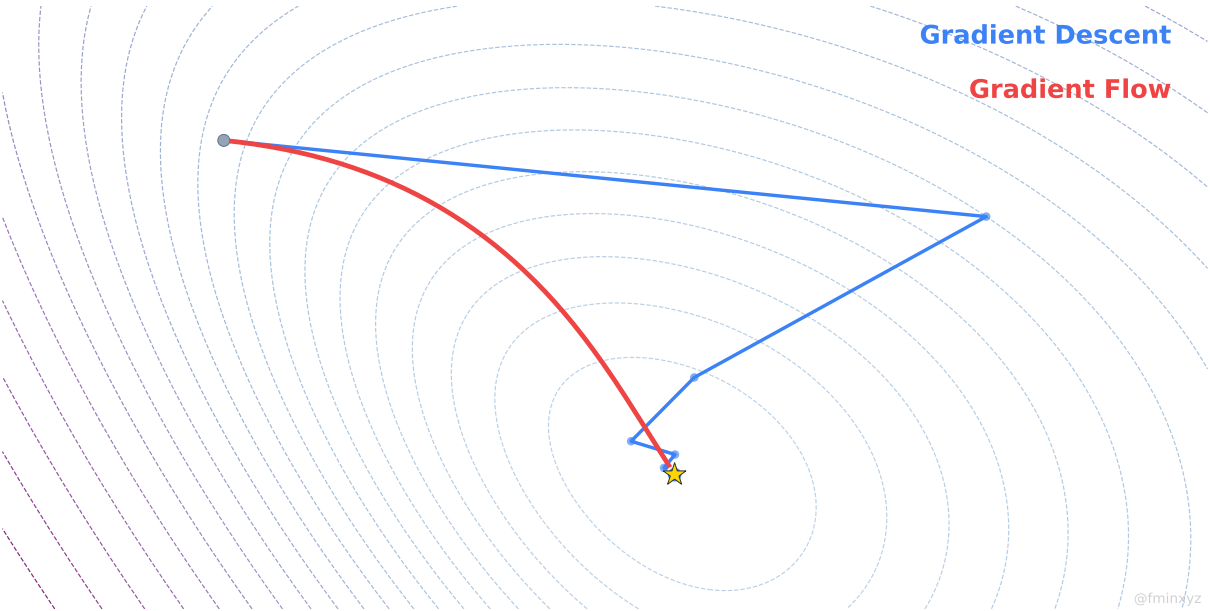
**Итог.** Решая QP в CVXPY, вы запускаете ADMM внутри OSQP; коническую задачу или SDP — ADMM внутри SCS. Канонический источник — монография Boyd, Parikh, Chu, Peleato, Eckstein (*Foundations and Trends in ML*, 2011), одна из самых цитируемых работ области.

## Непрерывные методы оптимизации



**Gradient Descent**

**Gradient Flow**



## Градиентный поток

## Интуиция градиентного потока

- Антиградиент  $-\nabla f(x)$  указывает направление **наискорейшего спуска** в точке  $x$ .

## Интуиция градиентного потока

- Антиградиент  $-\nabla f(x)$  указывает направление **наискорейшего спуска** в точке  $x$ .
- Антиградиент решает задачу минимизации линейного приближения Тейлора на евклидовом шаре:

$$\min_{\delta x \in \mathbb{R}^n} \nabla f(x_0)^\top \delta x$$

при  $\delta x^\top \delta x = \varepsilon^2$

## Интуиция градиентного потока

- Антиградиент  $-\nabla f(x)$  указывает направление **наискорейшего спуска** в точке  $x$ .
- Антиградиент решает задачу минимизации линейного приближения Тейлора на евклидовом шаре:

$$\min_{\delta x \in \mathbb{R}^n} \nabla f(x_0)^\top \delta x$$

при  $\delta x^\top \delta x = \varepsilon^2$

## Интуиция градиентного потока

- Антиградиент  $-\nabla f(x)$  указывает направление **наискорейшего спуска** в точке  $x$ .
- Антиградиент решает задачу минимизации линейного приближения Тейлора на евклидовом шаре:

$$\min_{\delta x \in \mathbb{R}^n} \nabla f(x_0)^\top \delta x$$

при  $\delta x^\top \delta x = \varepsilon^2$

- Метод градиентного спуска — итерационный алгоритм для минимизации дифференцируемых функций:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

## Интуиция градиентного потока

- Антиградиент  $-\nabla f(x)$  указывает направление **наискорейшего спуска** в точке  $x$ .
- Антиградиент решает задачу минимизации линейного приближения Тейлора на евклидовом шаре:

$$\min_{\delta x \in \mathbb{R}^n} \nabla f(x_0)^\top \delta x$$

при  $\delta x^\top \delta x = \varepsilon^2$

- Метод градиентного спуска — итерационный алгоритм для минимизации дифференцируемых функций:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

- Запишем это как разностную схему:

$$\frac{x_{k+1} - x_k}{\alpha_k} = -\nabla f(x_k)$$

## Интуиция градиентного потока

- Антиградиент  $-\nabla f(x)$  указывает направление **наискорейшего спуска** в точке  $x$ .
- Антиградиент решает задачу минимизации линейного приближения Тейлора на евклидовом шаре:

$$\min_{\delta x \in \mathbb{R}^n} \nabla f(x_0)^\top \delta x$$

при  $\delta x^\top \delta x = \varepsilon^2$

- Метод градиентного спуска — итерационный алгоритм для минимизации дифференцируемых функций:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

- Запишем это как разностную схему:

$$\frac{x_{k+1} - x_k}{\alpha_k} = -\nabla f(x_k)$$



## Интуиция градиентного потока

- Антиградиент  $-\nabla f(x)$  указывает направление **наискорейшего спуска** в точке  $x$ .
- Антиградиент решает задачу минимизации линейного приближения Тейлора на евклидовом шаре:

$$\min_{\delta x \in \mathbb{R}^n} \nabla f(x_0)^\top \delta x$$

при  $\delta x^\top \delta x = \varepsilon^2$

- Метод градиентного спуска — итерационный алгоритм для минимизации дифференцируемых функций:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

- Запишем это как разностную схему:

$$\frac{x_{k+1} - x_k}{\alpha_k} = -\nabla f(x_k)$$

**Градиентный поток** — это предел градиентного спуска при  $\alpha_k \rightarrow 0$ :

!

$$\frac{dx}{dt} = -\nabla f(x)$$

## Зачем изучать методы в непрерывном времени?

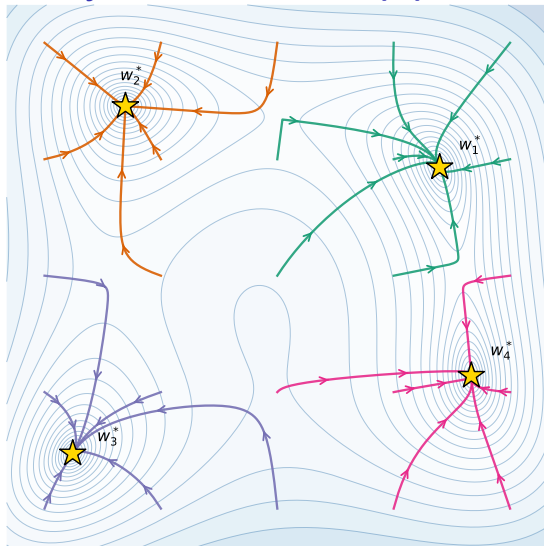


Figure 10: Анимация 

- **Упрощённый анализ.** Нет шага  $\alpha$  — исчезают все классические проблемы: линейный поиск, константный шаг, убывающие расписания.

## Зачем изучать методы в непрерывном времени?

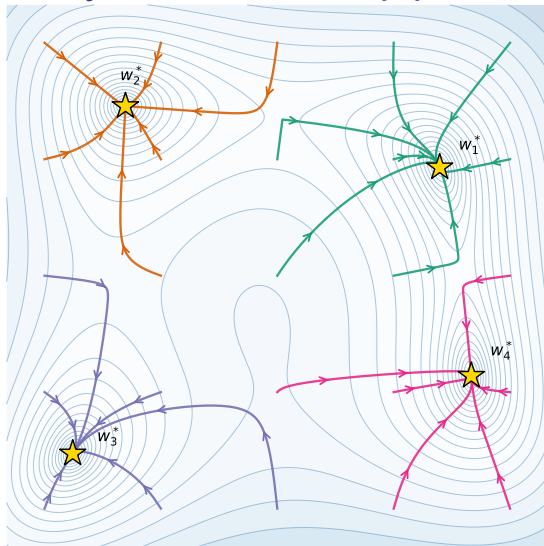


Figure 10: Анимация 

- **Упрощённый анализ.** Нет шага  $\alpha$  — исчезают все классические проблемы: линейный поиск, константный шаг, убывающие расписания.
- **Аналитическое решение.** Для квадратичных задач  $f(x) = \frac{1}{2}x^\top Ax$  градиент линеен, и ОДУ решается явно.

## Зачем изучать методы в непрерывном времени?

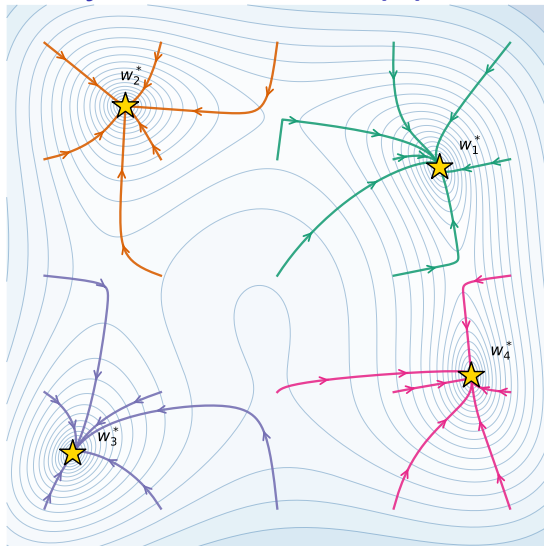


Figure 10: Анимация

- **Упрощённый анализ.** Нет шага  $\alpha$  — исчезают все классические проблемы: линейный поиск, константный шаг, убывающие расписания.
- **Аналитическое решение.** Для квадратичных задач  $f(x) = \frac{1}{2}x^\top Ax$  градиент линеен, и ОДУ решается явно.
- **Разные дискретизации  $\rightarrow$  разные методы.** Одно ОДУ порождает целое семейство алгоритмов: от классического GD до проксимальных и ускоренных методов.

## Зачем изучать методы в непрерывном времени?

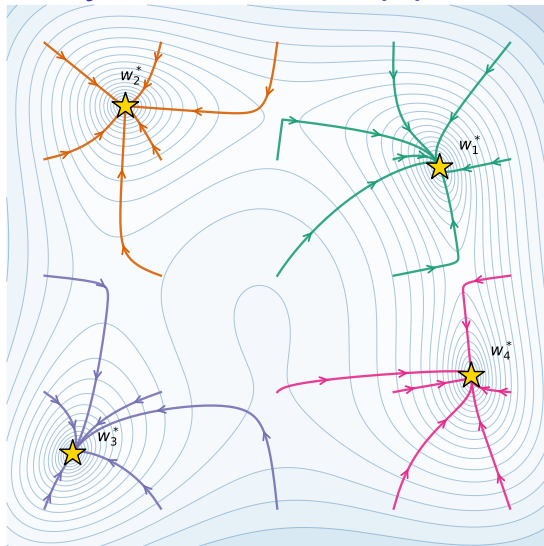


Figure 10: Анимация 

- **Упрощённый анализ.** Нет шага  $\alpha$  — исчезают все классические проблемы: линейный поиск, константный шаг, убывающие расписания.
- **Аналитическое решение.** Для квадратичных задач  $f(x) = \frac{1}{2}x^\top Ax$  градиент линеен, и ОДУ решается явно.
- **Разные дискретизации  $\rightarrow$  разные методы.** Одно ОДУ порождает целое семейство алгоритмов: от классического GD до проксимальных и ускоренных методов.
- **Слева:** 25 траекторий градиентного потока  $\dot{w} = -\nabla f(w)$  на функции Химмельблау из равномерно расположенных стартов. Цвет траектории — это её бассейн притяжения: невыпуклая задача, начальная точка определяет, в какой из 4 минимумов мы придём.

# Дискретизации градиентного потока

Рассмотрим ОДУ градиентного потока:

$$\frac{dx}{dt} = -\nabla f(x)$$

**Явный метод Эйлера:**

# Дискретизации градиентного потока

Рассмотрим ОДУ градиентного потока:

$$\frac{dx}{dt} = -\nabla f(x)$$

**Явный метод Эйлера:**

$$\frac{x_{k+1} - x_k}{\alpha} = -\nabla f(x_k)$$

Приводит к методу градиентного спуска:

# Дискретизации градиентного потока

Рассмотрим ОДУ градиентного потока:

$$\frac{dx}{dt} = -\nabla f(x)$$

**Явный метод Эйлера:**

$$\frac{x_{k+1} - x_k}{\alpha} = -\nabla f(x_k)$$

Приводит к методу градиентного спуска:

$$\boxed{x_{k+1} = x_k - \alpha \nabla f(x_k)} \quad (\text{GD})$$



# Дискретизации градиентного потока

Рассмотрим ОДУ градиентного потока:

$$\frac{dx}{dt} = -\nabla f(x)$$

**Явный метод Эйлера:**

$$\frac{x_{k+1} - x_k}{\alpha} = -\nabla f(x_k)$$

Приводит к методу градиентного спуска:

$$\boxed{x_{k+1} = x_k - \alpha \nabla f(x_k)}$$

**Неявный метод Эйлера:**

$$\frac{x_{k+1} - x_k}{\alpha} = -\nabla f(x_{k+1})$$

$$x_{k+1} = x_k - \alpha \nabla f(x_{k+1})$$

$\Updownarrow$

(GD)

$$x_{k+1} = \arg \min_x \left[ f(x) + \frac{1}{2\alpha} \|x - x_k\|_2^2 \right]$$

# Дискретизации градиентного потока

Рассмотрим ОДУ градиентного потока:

$$\frac{dx}{dt} = -\nabla f(x)$$

**Явный метод Эйлера:**

$$\frac{x_{k+1} - x_k}{\alpha} = -\nabla f(x_k)$$

Приводит к методу градиентного спуска:

$$\boxed{x_{k+1} = x_k - \alpha \nabla f(x_k)}$$

**Неявный метод Эйлера:**

$$\frac{x_{k+1} - x_k}{\alpha} = -\nabla f(x_{k+1})$$

$$x_{k+1} = x_k - \alpha \nabla f(x_{k+1})$$

$\Updownarrow$

$$x_{k+1} = \arg \min_x \left[ f(x) + \frac{1}{2\alpha} \|x - x_k\|_2^2 \right]$$

(GD)

$$\boxed{x_{k+1} = \text{prox}_{\alpha f}(x_k)}$$

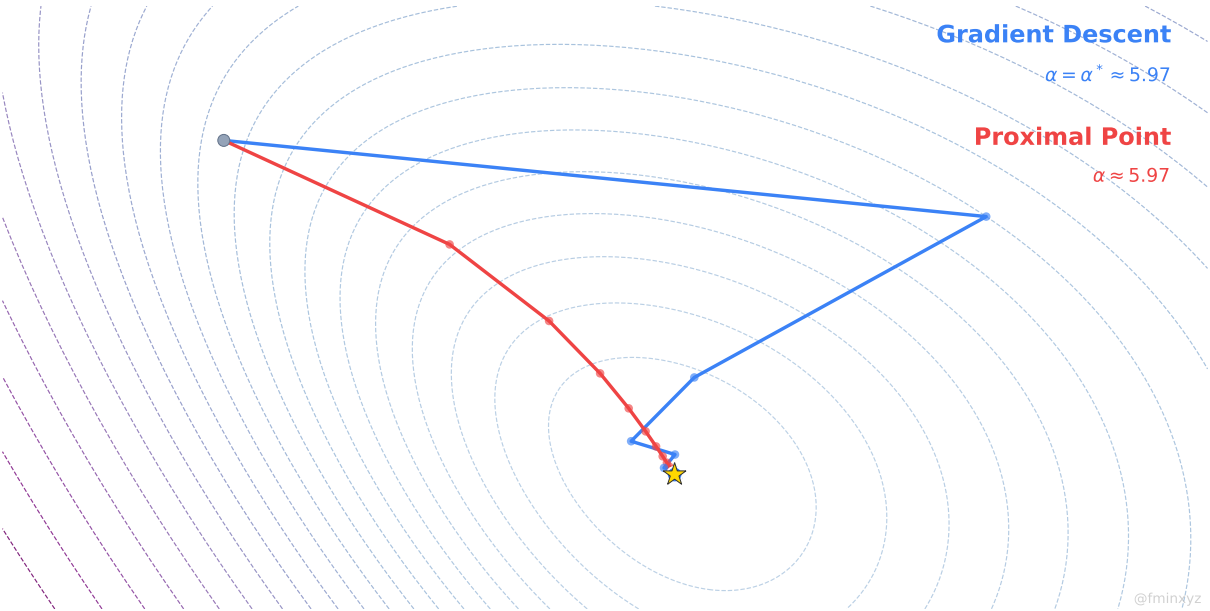
(PPM)

## Gradient Descent

$$\alpha = \alpha^* \approx 5.97$$

## Proximal Point

$$\alpha \approx 5.97$$

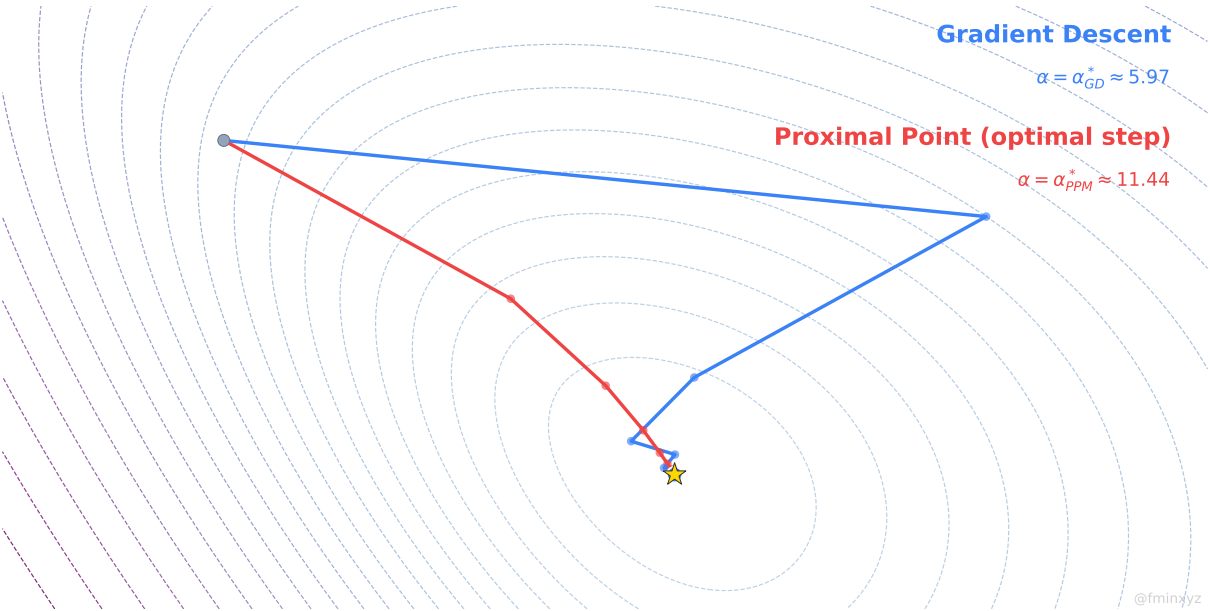


## Gradient Descent

$$\alpha = \alpha_{GD}^* \approx 5.97$$

**Proximal Point (optimal step)**

$$\alpha = \alpha_{PPM}^* \approx 11.44$$



## Анализ сходимости: выпуклый случай

**Шаг 1.** Монотонное убывание по градиентному потоку:

$$\frac{d}{dt}f(x(t)) = \nabla f(x(t))^\top \dot{x}(t) = -\|\nabla f(x(t))\|_2^2 \leq 0.$$

Если  $f$  ограничена снизу, значение  $f(x(t))$  имеет предел; это ещё не означает, что сама траектория  $x(t)$  сходится.

## Анализ сходимости: выпуклый случай

**Шаг 1.** Монотонное убывание по градиентному потоку:

$$\frac{d}{dt}f(x(t)) = \nabla f(x(t))^\top \dot{x}(t) = -\|\nabla f(x(t))\|_2^2 \leq 0.$$

Если  $f$  ограничена снизу, значение  $f(x(t))$  имеет предел; это ещё не означает, что сама траектория  $x(t)$  сходится.

**Шаг 2.** Если  $f$  дополнительно выпукла:

$$\frac{d}{dt}[\|x(t) - x^*\|^2] = -2(x(t) - x^*)^\top \nabla f(x(t)) \leq -2[f(x(t)) - f^*]$$

## Анализ сходимости: выпуклый случай

**Шаг 1.** Монотонное убывание по градиентному потоку:

$$\frac{d}{dt}f(x(t)) = \nabla f(x(t))^\top \dot{x}(t) = -\|\nabla f(x(t))\|_2^2 \leq 0.$$

Если  $f$  ограничена снизу, значение  $f(x(t))$  имеет предел; это ещё не означает, что сама траектория  $x(t)$  сходится.

**Шаг 2.** Если  $f$  дополнительно выпукла:

$$\frac{d}{dt}[\|x(t) - x^*\|^2] = -2(x(t) - x^*)^\top \nabla f(x(t)) \leq -2[f(x(t)) - f^*]$$

**Шаг 3.** Интегрируем от 0 до  $t$  и используем монотонность  $f(x(t))$ :

$$f(x(t)) - f^* \leq \frac{1}{t} \int_0^t [f(x(u)) - f^*] du \leq \frac{1}{2t} \|x(0) - x^*\|^2.$$

## Анализ сходимости: выпуклый случай

**Шаг 1.** Монотонное убывание по градиентному потоку:

$$\frac{d}{dt} f(x(t)) = \nabla f(x(t))^\top \dot{x}(t) = -\|\nabla f(x(t))\|_2^2 \leq 0.$$

Если  $f$  ограничена снизу, значение  $f(x(t))$  имеет предел; это ещё не означает, что сама траектория  $x(t)$  сходится.

**Шаг 2.** Если  $f$  дополнительно выпукла:

$$\frac{d}{dt} [\|x(t) - x^*\|^2] = -2(x(t) - x^*)^\top \nabla f(x(t)) \leq -2[f(x(t)) - f^*]$$

**Шаг 3.** Интегрируем от 0 до  $t$  и используем монотонность  $f(x(t))$ :

$$f(x(t)) - f^* \leq \frac{1}{t} \int_0^t [f(x(u)) - f^*] du \leq \frac{1}{2t} \|x(0) - x^*\|^2.$$



$$f(x(t)) - f^* \leq \frac{\|x(0) - x^*\|^2}{2t} = \mathcal{O}\left(\frac{1}{t}\right)$$



## Анализ сходимости: условие PL

**Условие Поляка–Лоясевича (PL):** функция  $f$  удовлетворяет PL-условию, если

$$\|\nabla f(x)\|^2 \geq 2\mu(f(x) - f^*) \quad \forall x.$$

## Анализ сходимости: условие PL

**Условие Поляка–Лоясевича (PL):** функция  $f$  удовлетворяет PL-условию, если

$$\|\nabla f(x)\|^2 \geq 2\mu(f(x) - f^*) \quad \forall x.$$

**Анализ.** Используем убывание целевой функции вдоль потока:

$$\frac{d}{dt}[f(x(t)) - f^*] = \nabla f(x(t))^\top \dot{x}(t) = -\|\nabla f(x(t))\|_2^2 \leq -2\mu[f(x(t)) - f^*]$$

## Анализ сходимости: условие PL

**Условие Поляка–Лоясевича (PL):** функция  $f$  удовлетворяет PL-условию, если

$$\|\nabla f(x)\|^2 \geq 2\mu(f(x) - f^*) \quad \forall x.$$

**Анализ.** Используем убывание целевой функции вдоль потока:

$$\frac{d}{dt}[f(x(t)) - f^*] = \nabla f(x(t))^\top \dot{x}(t) = -\|\nabla f(x(t))\|_2^2 \leq -2\mu[f(x(t)) - f^*]$$

**Теорема Гронуолла** немедленно даёт:



$$f(x(t)) - f^* \leq e^{-2\mu t}[f(x(0)) - f^*]$$

## Анализ сходимости: условие PL

**Условие Поляка–Лоясевича (PL):** функция  $f$  удовлетворяет PL-условию, если

$$\|\nabla f(x)\|^2 \geq 2\mu(f(x) - f^*) \quad \forall x.$$

**Анализ.** Используем убывание целевой функции вдоль потока:

$$\frac{d}{dt}[f(x(t)) - f^*] = \nabla f(x(t))^\top \dot{x}(t) = -\|\nabla f(x(t))\|_2^2 \leq -2\mu[f(x(t)) - f^*]$$

**Теорема Гронуолла** немедленно даёт:



$$f(x(t)) - f^* \leq e^{-2\mu t}[f(x(0)) - f^*]$$

Это **экспоненциальная сходимость** — непрерывный аналог линейной сходимости GD для сильно выпуклых функций.

## Анализ сходимости: два базовых результата

**i** Градиентный поток на выпуклой функции.

Пусть  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  — выпуклая  $C^1$ -функция,  $\arg \min f \neq \emptyset$ ,  $x^* \in \arg \min f$ ,  $f^* = f(x^*)$ . Если  $x : [0, \infty) \rightarrow \mathbb{R}^n$  — глобальное  $C^1$ -решение градиентного потока

$$\dot{x}(t) = -\nabla f(x(t)), \quad x(0) = x_0,$$

то для любого  $t > 0$

$$f(x(t)) - f^* \leq \frac{\|x_0 - x^*\|^2}{2t}.$$

**i** Градиентный поток при условии PL.

Пусть  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  —  $C^1$ -функция,  $f^* = \inf_x f(x) > -\infty$ , и для некоторого  $\mu > 0$  выполнено условие Поляка - Лоясиевича. Если  $x : [0, \infty) \rightarrow \mathbb{R}^n$  — глобальное  $C^1$ -решение  $\dot{x}(t) = -\nabla f(x(t))$ , то

$$f(x(t)) - f^* \leq e^{-2\mu t} (f(x_0) - f^*) \quad \forall t \geq 0.$$

## Ускоренный градиентный поток

Вспомним convex-smooth форму ускоренного метода Нестерова (NAG):

$$\begin{aligned}x_{k+1} &= y_k - \alpha \nabla f(y_k) \\ y_k &= x_k + \frac{k-1}{k+2}(x_k - x_{k-1})\end{aligned}$$

---

<sup>1</sup>A Differential Equation for Modeling Nesterov's Accelerated Gradient Method: Theory and Insights, Su, Boyd, Candes, 2016

Вспомним convex-smooth форму ускоренного метода Нестерова (NAG):

$$\begin{aligned}x_{k+1} &= y_k - \alpha \nabla f(y_k) \\ y_k &= x_k + \frac{k-1}{k+2}(x_k - x_{k-1})\end{aligned}$$

Соответствующее <sup>1</sup> ОДУ непрерывного времени:

**! ОДУ Нестерова (AGF)**

$$\ddot{x}(t) + \frac{3}{t}\dot{x}(t) + \nabla f(x(t)) = 0$$

<sup>1</sup>A Differential Equation for Modeling Nesterov's Accelerated Gradient Method: Theory and Insights, Su, Boyd, Candes, 2016



Вспомним convex-smooth форму ускоренного метода Нестерова (NAG):

$$\begin{aligned}x_{k+1} &= y_k - \alpha \nabla f(y_k) \\ y_k &= x_k + \frac{k-1}{k+2}(x_k - x_{k-1})\end{aligned}$$

Соответствующее <sup>1</sup> ОДУ непрерывного времени:

! ОДУ Нестерова (AGF)

$$\ddot{x}(t) + \frac{3}{t}\dot{x}(t) + \nabla f(x(t)) = 0$$

Интерпретация: это уравнение **затухающего осциллятора**, где коэффициент демпфирования  $3/t$  убывает со временем. При малых  $t$  трение велико и осцилляций нет; с ростом  $t$  трение убывает, что и обеспечивает ускорение.

---

<sup>1</sup>A Differential Equation for Modeling Nesterov's Accelerated Gradient Method: Theory and Insights, Su, Boyd, Candes, 2016

## Вывод ОДУ Нестерова: дискретная алгебра

Обозначим

$$\beta_k = \frac{k-1}{k+2} = 1 - \frac{3}{k+2}.$$

# Вывод ОДУ Нестерова: дискретная алгебра

Обозначим

$$\beta_k = \frac{k-1}{k+2} = 1 - \frac{3}{k+2}.$$

Схема NAG

$$\begin{aligned} y_k &= x_k + \beta_k(x_k - x_{k-1}), \\ x_{k+1} &= y_k - \alpha \nabla f(y_k). \end{aligned}$$

Собираем в одно уравнение

$$x_{k+1} = x_k + \beta_k(x_k - x_{k-1}) - \alpha \nabla f(y_k).$$

## Вывод ОДУ Нестерова: дискретная алгебра

Обозначим

$$\beta_k = \frac{k-1}{k+2} = 1 - \frac{3}{k+2}.$$

Схема NAG

Собираем в одно уравнение

$$\begin{aligned} y_k &= x_k + \beta_k(x_k - x_{k-1}), \\ x_{k+1} &= y_k - \alpha \nabla f(y_k). \end{aligned}$$

$$x_{k+1} = x_k + \beta_k(x_k - x_{k-1}) - \alpha \nabla f(y_k).$$

Вычитаем из нового шага предыдущий:

$$(x_{k+1} - x_k) - (x_k - x_{k-1}).$$

Это изменение приращения за один шаг — дискретный аналог ускорения.

## Вывод ОДУ Нестерова: дискретная алгебра

Обозначим

$$\beta_k = \frac{k-1}{k+2} = 1 - \frac{3}{k+2}.$$

Схема NAG

Собираем в одно уравнение

$$\begin{aligned} y_k &= x_k + \beta_k(x_k - x_{k-1}), \\ x_{k+1} &= y_k - \alpha \nabla f(y_k). \end{aligned}$$

$$x_{k+1} = x_k + \beta_k(x_k - x_{k-1}) - \alpha \nabla f(y_k).$$

Вычитаем из нового шага предыдущий:

$$(x_{k+1} - x_k) - (x_k - x_{k-1}).$$

Это изменение приращения за один шаг — дискретный аналог ускорения.

$$\underbrace{x_{k+1} - 2x_k + x_{k-1}}_{\text{изменение шага}} = - \underbrace{\frac{3}{k+2}(x_k - x_{k-1})}_{\text{затухающая инерция}} - \underbrace{\alpha \nabla f(y_k)}_{\text{градиентный шаг}}.$$

После деления на квадрат масштаба времени левая часть станет  $\ddot{x}(t)$ , а затухающая инерция справа превратится в трение  $3\dot{x}(t)/t$ .

## Вывод ОДУ Нестерова: масштаб времени

Положим  $t_k = kh$ ,  $x(t_k) = x_k$ . Тогда по Тейлору около  $t_k$ :

$$x_{k+1} - x_k = x(t_k + h) - x(t_k) = h\dot{x}(t_k) + \frac{h^2}{2}\ddot{x}(t_k) + \mathcal{O}(h^3),$$

$$x_k - x_{k-1} = x(t_k) - x(t_k - h) = h\dot{x}(t_k) - \frac{h^2}{2}\ddot{x}(t_k) + \mathcal{O}(h^3).$$

## Вывод ОДУ Нестерова: масштаб времени

Положим  $t_k = kh$ ,  $x(t_k) = x_k$ . Тогда по Тейлору около  $t_k$ :

$$x_{k+1} - x_k = x(t_k + h) - x(t_k) = h\dot{x}(t_k) + \frac{h^2}{2}\ddot{x}(t_k) + \mathcal{O}(h^3),$$

$$x_k - x_{k-1} = x(t_k) - x(t_k - h) = h\dot{x}(t_k) - \frac{h^2}{2}\ddot{x}(t_k) + \mathcal{O}(h^3).$$

Вычитаем соседние шаги:

$$(x_{k+1} - x_k) - (x_k - x_{k-1}) = h^2\ddot{x}(t_k) + \mathcal{O}(h^3).$$

Значит, чтобы получить конечный непрерывный предел, нужно делить на  $h^2$ . Градиентный член в NAG равен  $\alpha \nabla f(y_k)$ , поэтому выбираем

$$\alpha = h^2, \quad h = \sqrt{\alpha}.$$

## Вывод ОДУ Нестерова: непрерывный предел

Положим  $h = \sqrt{\alpha}$ ,  $t_k = kh$ ,  $x(t_k) = x_k$ .



## Вывод ОДУ Нестерова: непрерывный предел

Положим  $h = \sqrt{\alpha}$ ,  $t_k = kh$ ,  $x(t_k) = x_k$ .

Делим дискретное уравнение на  $h^2$ :

$$\frac{x_{k+1} - 2x_k + x_{k-1}}{h^2} + \frac{3}{(k+2)h} \frac{x_k - x_{k-1}}{h} + \nabla f(y_k) = 0.$$

## Вывод ОДУ Нестерова: непрерывный предел

Положим  $h = \sqrt{\alpha}$ ,  $t_k = kh$ ,  $x(t_k) = x_k$ .

Делим дискретное уравнение на  $h^2$ :

$$\frac{x_{k+1} - 2x_k + x_{k-1}}{h^2} + \frac{3}{(k+2)h} \frac{x_k - x_{k-1}}{h} + \nabla f(y_k) = 0.$$

Теперь при  $h \rightarrow 0$  и  $t_k \rightarrow t$ :

$$\begin{aligned} \frac{x_{k+1} - 2x_k + x_{k-1}}{h^2} &\rightarrow \ddot{x}(t), & \frac{x_k - x_{k-1}}{h} &\rightarrow \dot{x}(t), \\ (k+2)h = t_k + 2h &\rightarrow t, & y_k &\rightarrow x(t). \end{aligned}$$

## Вывод ОДУ Нестерова: непрерывный предел

Положим  $h = \sqrt{\alpha}$ ,  $t_k = kh$ ,  $x(t_k) = x_k$ .

Делим дискретное уравнение на  $h^2$ :

$$\frac{x_{k+1} - 2x_k + x_{k-1}}{h^2} + \frac{3}{(k+2)h} \frac{x_k - x_{k-1}}{h} + \nabla f(y_k) = 0.$$

Теперь при  $h \rightarrow 0$  и  $t_k \rightarrow t$ :

$$\begin{aligned} \frac{x_{k+1} - 2x_k + x_{k-1}}{h^2} &\rightarrow \ddot{x}(t), & \frac{x_k - x_{k-1}}{h} &\rightarrow \dot{x}(t), \\ (k+2)h = t_k + 2h &\rightarrow t, & y_k &\rightarrow x(t). \end{aligned}$$

Значит,



$$\ddot{x}(t) + \frac{3}{t}\dot{x}(t) + \nabla f(x(t)) = 0$$

Коэффициент  $\frac{3}{k+2}$  в дискретном времени после деления на  $h^2$  даёт множитель  $\frac{3}{(k+2)h} \rightarrow \frac{3}{t}$ .

## Функционал Ляпунова: что надо доказать

Хотим получить ускоренную оценку

$$f(x(t)) - f^* = \mathcal{O}\left(\frac{1}{t^2}\right).$$

Для этого достаточно построить функционал Ляпунова  $E(t)$ , который не возрастает вдоль траектории и мажорирует  $t^2(f(x(t)) - f^*)$ .

## Функционал Ляпунова: что надо доказать

Хотим получить ускоренную оценку

$$f(x(t)) - f^* = \mathcal{O}\left(\frac{1}{t^2}\right).$$

Для этого достаточно построить функционал Ляпунова  $E(t)$ , который не возрастает вдоль траектории и мажорирует  $t^2(f(x(t)) - f^*)$ .

Обозначим  $r = x(t) - x^*$  и

$$v = r + \frac{t}{2}\dot{x}(t).$$

## Функционал Ляпунова: что надо доказать

Хотим получить ускоренную оценку

$$f(x(t)) - f^* = \mathcal{O}\left(\frac{1}{t^2}\right).$$

Для этого достаточно построить функционал Ляпунова  $E(t)$ , который не возрастает вдоль траектории и мажорирует  $t^2(f(x(t)) - f^*)$ .

Обозначим  $r = x(t) - x^*$  и

$$v = r + \frac{t}{2}\dot{x}(t).$$

!

$$E(t) = t^2(f(x(t)) - f^*) + 2\|v\|^2.$$

## Функционал Ляпунова: что надо доказать

Хотим получить ускоренную оценку

$$f(x(t)) - f^* = \mathcal{O}\left(\frac{1}{t^2}\right).$$

Для этого достаточно построить функционал Ляпунова  $E(t)$ , который не возрастает вдоль траектории и мажорирует  $t^2(f(x(t)) - f^*)$ .

Обозначим  $r = x(t) - x^*$  и

$$v = r + \frac{t}{2}\dot{x}(t).$$



$$E(t) = t^2(f(x(t)) - f^*) + 2\|v\|^2.$$

Если покажем  $\dot{E}(t) \leq 0$ , то сразу

$$t^2(f(x(t)) - f^*) \leq E(t) \leq E(0).$$

# Функционал Ляпунова: вычисление $\dot{v}$

Вычислим  $\dot{v}$ . Из ОДУ Нестерова

$$\ddot{x}(t) = -\frac{3}{t}\dot{x}(t) - \nabla f(x(t)).$$



## Функционал Ляпунова: вычисление $\dot{v}$

Вычислим  $\dot{v}$ . Из ОДУ Нестерова

$$\ddot{x}(t) = -\frac{3}{t}\dot{x}(t) - \nabla f(x(t)).$$

Тогда

$$\begin{aligned}\dot{v} &= \dot{x}(t) + \frac{1}{2}\dot{x}(t) + \frac{t}{2}\ddot{x}(t) \\ &= \frac{3}{2}\dot{x}(t) + \frac{t}{2}\left(-\frac{3}{t}\dot{x}(t) - \nabla f(x(t))\right) \\ &= -\frac{t}{2}\nabla f(x(t)).\end{aligned}$$

## Функционал Ляпунова: вычисление $\dot{v}$

Вычислим  $\dot{v}$ . Из ОДУ Нестерова

$$\ddot{x}(t) = -\frac{3}{t}\dot{x}(t) - \nabla f(x(t)).$$

Тогда

$$\begin{aligned}\dot{v} &= \dot{x}(t) + \frac{1}{2}\dot{x}(t) + \frac{t}{2}\ddot{x}(t) \\ &= \frac{3}{2}\dot{x}(t) + \frac{t}{2}\left(-\frac{3}{t}\dot{x}(t) - \nabla f(x(t))\right) \\ &= -\frac{t}{2}\nabla f(x(t)).\end{aligned}$$

Такой выбор  $v = r + \frac{t}{2}\dot{x}$  удобен потому, что слагаемые с  $\dot{x}$  сокращаются благодаря коэффициенту  $3/t$ .

## Функционал Ляпунова: производная энергии

Для краткости пишем  $f = f(x(t))$ ,  $\nabla f = \nabla f(x(t))$ ,  $r = x(t) - x^*$ .

## Функционал Ляпунова: производная энергии

Для краткости пишем  $f = f(x(t))$ ,  $\nabla f = \nabla f(x(t))$ ,  $r = x(t) - x^*$ .

Первая часть:

$$\frac{d}{dt} [t^2(f - f^*)] = 2t(f - f^*) + t^2 \nabla f^\top \dot{x}.$$

## Функционал Ляпунова: производная энергии

Для краткости пишем  $f = f(x(t))$ ,  $\nabla f = \nabla f(x(t))$ ,  $r = x(t) - x^*$ .

Первая часть:

$$\frac{d}{dt} [t^2(f - f^*)] = 2t(f - f^*) + t^2 \nabla f^\top \dot{x}.$$

Вторая часть:

$$\frac{d}{dt} [2\|v\|^2] = 4v^\top \dot{v} = -2t \left( r + \frac{t}{2} \dot{x} \right)^\top \nabla f = -2t r^\top \nabla f - t^2 \nabla f^\top \dot{x}.$$

## Функционал Ляпунова: производная энергии

Для краткости пишем  $f = f(x(t))$ ,  $\nabla f = \nabla f(x(t))$ ,  $r = x(t) - x^*$ .

Первая часть:

$$\frac{d}{dt} [t^2(f - f^*)] = 2t(f - f^*) + t^2 \nabla f^\top \dot{x}.$$

Вторая часть:

$$\frac{d}{dt} [2\|v\|^2] = 4v^\top \dot{v} = -2t \left( r + \frac{t}{2} \dot{x} \right)^\top \nabla f = -2t r^\top \nabla f - t^2 \nabla f^\top \dot{x}.$$

Складываем: смешанные члены  $t^2 \nabla f^\top \dot{x}$  сокращаются,

$$\dot{E}(t) = 2t [(f - f^*) - r^\top \nabla f].$$

# Функционал Ляпунова: выпуклость закрывает доказательство

По выпуклости:

$$f(x^*) \geq f(x(t)) + \nabla f(x(t))^\top (x^* - x(t)).$$

Эквивалентно,

$$r^\top \nabla f(x(t)) = (x(t) - x^*)^\top \nabla f(x(t)) \geq f(x(t)) - f^*.$$

# Функционал Ляпунова: выпуклость закрывает доказательство

По выпуклости:

$$f(x^*) \geq f(x(t)) + \nabla f(x(t))^\top (x^* - x(t)).$$

Эквивалентно,

$$r^\top \nabla f(x(t)) = (x(t) - x^*)^\top \nabla f(x(t)) \geq f(x(t)) - f^*.$$

!

$$\dot{E}(t) = 2t \underbrace{[(f - f^*) - r^\top \nabla f]}_{\leq 0} \leq 0.$$



## Функционал Ляпунова: выпуклость закрывает доказательство

По выпуклости:

$$f(x^*) \geq f(x(t)) + \nabla f(x(t))^\top (x^* - x(t)).$$

Эквивалентно,

$$r^\top \nabla f(x(t)) = (x(t) - x^*)^\top \nabla f(x(t)) \geq f(x(t)) - f^*.$$

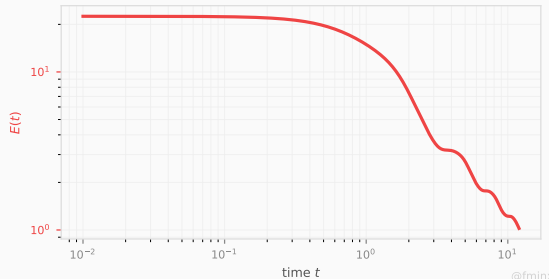
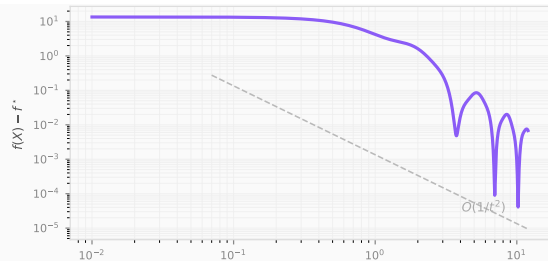
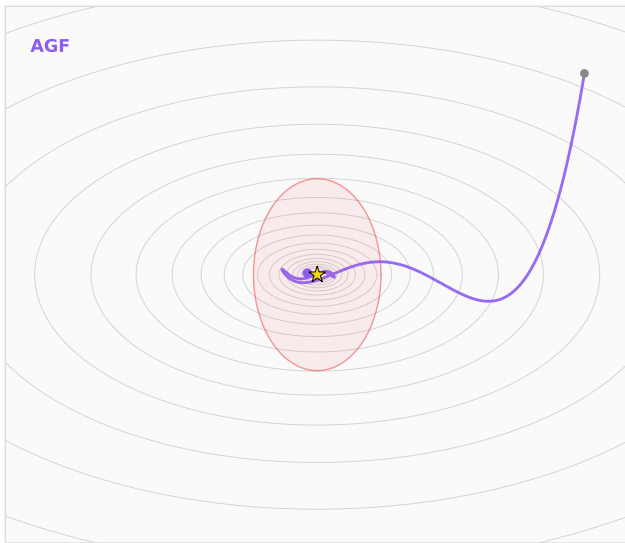
!

$$\dot{E}(t) = 2t \underbrace{[(f - f^*) - r^\top \nabla f]}_{\leq 0} \leq 0.$$

Значит  $E(t) \leq E(0)$ . При  $t = 0$  имеем  $v(0) = x_0 - x^*$ , поэтому  $E(0) = 2\|x_0 - x^*\|^2$ , и

$$f(x(t)) - f^* \leq \frac{E(0)}{t^2} = \frac{2\|x_0 - x^*\|^2}{t^2}.$$

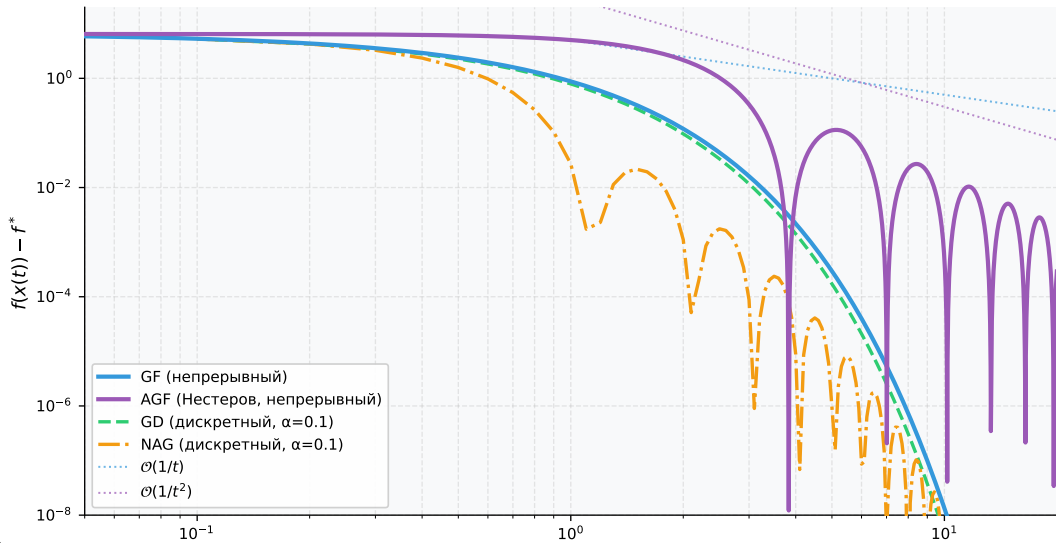
# Функционал Ляпунова: визуализация



# Сравнение скоростей сходимости

Скорости сходимости: GF, AGF и дискретные аналоги

$$f(x_1, x_2) = \frac{1}{2}(x_1^2 + x_2^2)$$



## Сильно выпуклый случай

## AGF не использует сильную выпуклость

Затухание  $\frac{3}{t}$  стремится к нулю — при  $t \rightarrow \infty$  система ведёт себя как **незатухающий** осциллятор  $\ddot{x} + \nabla f = 0$ .

## AGF не использует сильную выпуклость

Затухание  $\frac{3}{t}$  стремится к нулю — при  $t \rightarrow \infty$  система ведёт себя как **незатухающий** осциллятор  $\ddot{x} + \nabla f = 0$ .

AGF достигает  $\mathcal{O}(1/t^2)$  **даже для сильно выпуклых функций** — сильная выпуклость не помогает.

## AGF не использует сильную выпуклость

Затухание  $\frac{3}{t}$  стремится к нулю — при  $t \rightarrow \infty$  система ведёт себя как **незатухающий** осциллятор  $\ddot{x} + \nabla f = 0$ .

AGF достигает  $\mathcal{O}(1/t^2)$  **даже для сильно выпуклых функций** — сильная выпуклость не помогает.

! Для  $\mu$ -сильно выпуклой  $f$  правильное ОДУ — с **постоянным затуханием**:

$$\ddot{x}(t) + 2\sqrt{\mu}\dot{x}(t) + \nabla f(x(t)) = 0.$$

Физическая аналогия: это пружина жёсткостью  $\mu$  с **критически затухающим** трением  $\gamma = 2\sqrt{\mu}$ .

# Функционал Ляпунова SC: что надо доказать

Хотим получить **линейную** оценку

$$f(x(t)) - f^* = \mathcal{O}(e^{-\sqrt{\mu}t}).$$

Обозначим  $r = x(t) - x^*$  и

$$z = \dot{x}(t) + \sqrt{\mu}r.$$



## Функционал Ляпунова SC: что надо доказать

Хотим получить **линейную** оценку

$$f(x(t)) - f^* = \mathcal{O}(e^{-\sqrt{\mu}t}).$$

Обозначим  $r = x(t) - x^*$  и

$$z = \dot{x}(t) + \sqrt{\mu}r.$$

!

$$E(t) = (f(x(t)) - f^*) + \frac{1}{2}\|z\|^2.$$

## Функционал Ляпунова SC: что надо доказать

Хотим получить **линейную** оценку

$$f(x(t)) - f^* = \mathcal{O}(e^{-\sqrt{\mu}t}).$$

Обозначим  $r = x(t) - x^*$  и

$$z = \dot{x}(t) + \sqrt{\mu}r.$$

!

$$E(t) = (f(x(t)) - f^*) + \frac{1}{2}\|z\|^2.$$

Если покажем  $\dot{E}(t) \leq -\sqrt{\mu}E(t)$ , то по лемме Гронуолла

$$f(x(t)) - f^* \leq E(t) \leq E(0) e^{-\sqrt{\mu}t},$$

где  $E(0) = f(x_0) - f^* + \frac{\mu}{2}\|x_0 - x^*\|^2$ .

## Функционал Ляпунова SC: вычисление $\dot{z}$

Из ОДУ с постоянным затуханием:

$$\ddot{x}(t) = -2\sqrt{\mu} \dot{x}(t) - \nabla f(x(t)).$$

## Функционал Ляпунова SC: вычисление $\dot{z}$

Из ОДУ с постоянным затуханием:

$$\ddot{x}(t) = -2\sqrt{\mu}\dot{x}(t) - \nabla f(x(t)).$$

$$\dot{z} = \ddot{x} + \sqrt{\mu}\dot{x} = -2\sqrt{\mu}\dot{x} - \nabla f + \sqrt{\mu}\dot{x} = -\sqrt{\mu}\dot{x} - \nabla f.$$

## Функционал Ляпунова SC: вычисление $\dot{z}$

Из ОДУ с постоянным затуханием:

$$\ddot{x}(t) = -2\sqrt{\mu}\dot{x}(t) - \nabla f(x(t)).$$

$$\dot{z} = \ddot{x} + \sqrt{\mu}\dot{x} = -2\sqrt{\mu}\dot{x} - \nabla f + \sqrt{\mu}\dot{x} = -\sqrt{\mu}\dot{x} - \nabla f.$$

Поскольку  $\dot{x} = z - \sqrt{\mu}r$ :

$$\dot{z} = -\sqrt{\mu}z + \mu r - \nabla f.$$

Члены с  $\dot{x}$  сокращаются благодаря коэффициенту  $2\sqrt{\mu}$  — так же, как в выпуклом случае члены сокращались благодаря  $3/t$ .

## Функционал Ляпунова SC: производная энергии

Пишем  $f = f(x(t))$ ,  $\nabla f = \nabla f(x(t))$ ,  $r = x(t) - x^*$ .

## Функционал Ляпунова SC: производная энергии

Пишем  $f = f(x(t))$ ,  $\nabla f = \nabla f(x(t))$ ,  $r = x(t) - x^*$ .

Первая часть:

$$\frac{d}{dt}(f - f^*) = \langle \nabla f, \dot{x} \rangle.$$

## Функционал Ляпунова SC: производная энергии

Пишем  $f = f(x(t))$ ,  $\nabla f = \nabla f(x(t))$ ,  $r = x(t) - x^*$ .

Первая часть:

$$\frac{d}{dt}(f - f^*) = \langle \nabla f, \dot{x} \rangle.$$

Вторая часть:

$$\frac{d}{dt} \left[ \frac{1}{2} \|z\|^2 \right] = \langle z, \dot{z} \rangle = \langle \dot{x} + \sqrt{\mu} r, -\sqrt{\mu} \dot{x} - \nabla f \rangle = -\sqrt{\mu} \|\dot{x}\|^2 - \langle \dot{x}, \nabla f \rangle - \mu \langle r, \dot{x} \rangle - \sqrt{\mu} \langle r, \nabla f \rangle.$$



## Функционал Ляпунова SC: производная энергии

Пишем  $f = f(x(t))$ ,  $\nabla f = \nabla f(x(t))$ ,  $r = x(t) - x^*$ .

Первая часть:

$$\frac{d}{dt}(f - f^*) = \langle \nabla f, \dot{x} \rangle.$$

Вторая часть:

$$\frac{d}{dt} \left[ \frac{1}{2} \|z\|^2 \right] = \langle z, \dot{z} \rangle = \langle \dot{x} + \sqrt{\mu} r, -\sqrt{\mu} \dot{x} - \nabla f \rangle = -\sqrt{\mu} \|\dot{x}\|^2 - \langle \dot{x}, \nabla f \rangle - \mu \langle r, \dot{x} \rangle - \sqrt{\mu} \langle r, \nabla f \rangle.$$

Складываем: члены  $\langle \nabla f, \dot{x} \rangle$  сокращаются,

$$\dot{E}(t) = -\sqrt{\mu} \|\dot{x}\|^2 - \mu \langle r, \dot{x} \rangle - \sqrt{\mu} \langle r, \nabla f \rangle.$$

## Функционал Ляпунова SC: сильная выпуклость закрывает доказательство

Проверим, что  $\dot{E} + \sqrt{\mu} E \leq 0$ .

## Функционал Ляпунова SC: сильная выпуклость закрывает доказательство

Проверим, что  $\dot{E} + \sqrt{\mu} E \leq 0$ .

Раскрываем  $\|z\|^2 = \|\dot{x}\|^2 + 2\sqrt{\mu}\langle \dot{x}, r \rangle + \mu\|r\|^2$ :

$$\dot{E} + \sqrt{\mu} E = -\frac{\sqrt{\mu}}{2}\|\dot{x}\|^2 + \sqrt{\mu}[(f - f^*) - \langle r, \nabla f \rangle] + \frac{\mu\sqrt{\mu}}{2}\|r\|^2.$$

## Функционал Ляпунова SC: сильная выпуклость закрывает доказательство

Проверим, что  $\dot{E} + \sqrt{\mu} E \leq 0$ .

Раскрываем  $\|z\|^2 = \|\dot{x}\|^2 + 2\sqrt{\mu}\langle\dot{x}, r\rangle + \mu\|r\|^2$ :

$$\dot{E} + \sqrt{\mu} E = -\frac{\sqrt{\mu}}{2}\|\dot{x}\|^2 + \sqrt{\mu}[(f - f^*) - \langle r, \nabla f \rangle] + \frac{\mu\sqrt{\mu}}{2}\|r\|^2.$$

По  $\mu$ -сильной выпуклости:

$$\langle r, \nabla f \rangle \geq (f - f^*) + \frac{\mu}{2}\|r\|^2.$$

## Функционал Ляпунова SC: сильная выпуклость закрывает доказательство

Проверим, что  $\dot{E} + \sqrt{\mu} E \leq 0$ .

Раскрываем  $\|z\|^2 = \|\dot{x}\|^2 + 2\sqrt{\mu}\langle\dot{x}, r\rangle + \mu\|r\|^2$ :

$$\dot{E} + \sqrt{\mu} E = -\frac{\sqrt{\mu}}{2}\|\dot{x}\|^2 + \sqrt{\mu}[(f - f^*) - \langle r, \nabla f \rangle] + \frac{\mu\sqrt{\mu}}{2}\|r\|^2.$$

По  $\mu$ -сильной выпуклости:

$$\langle r, \nabla f \rangle \geq (f - f^*) + \frac{\mu}{2}\|r\|^2.$$

Члены с  $\|r\|^2$  сокращаются точно:

$$\dot{E} + \sqrt{\mu} E \leq -\frac{\sqrt{\mu}}{2}\|\dot{x}\|^2 \leq 0.$$

## Функционал Ляпунова SC: сильная выпуклость закрывает доказательство

Проверим, что  $\dot{E} + \sqrt{\mu} E \leq 0$ .

Раскрываем  $\|z\|^2 = \|\dot{x}\|^2 + 2\sqrt{\mu}\langle\dot{x}, r\rangle + \mu\|r\|^2$ :

$$\dot{E} + \sqrt{\mu} E = -\frac{\sqrt{\mu}}{2}\|\dot{x}\|^2 + \sqrt{\mu}[(f - f^*) - \langle r, \nabla f \rangle] + \frac{\mu\sqrt{\mu}}{2}\|r\|^2.$$

По  $\mu$ -сильной выпуклости:

$$\langle r, \nabla f \rangle \geq (f - f^*) + \frac{\mu}{2}\|r\|^2.$$

Члены с  $\|r\|^2$  сокращаются точно:

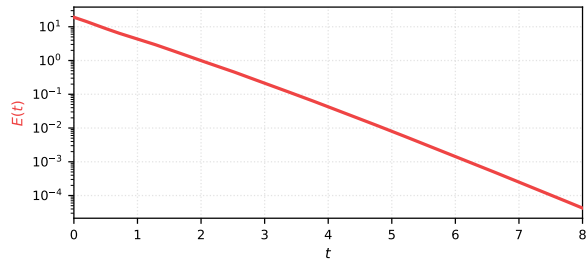
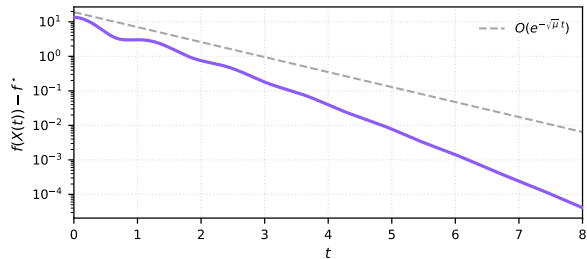
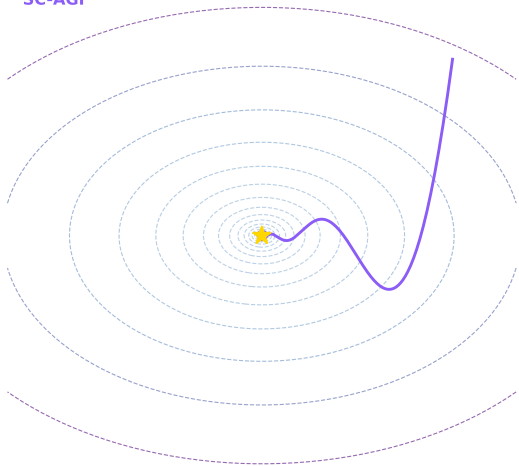
$$\dot{E} + \sqrt{\mu} E \leq -\frac{\sqrt{\mu}}{2}\|\dot{x}\|^2 \leq 0.$$

!

$$f(x(t)) - f^* \leq E(0) e^{-\sqrt{\mu}t} = \left(f(x_0) - f^* + \frac{\mu}{2}\|x_0 - x^*\|^2\right) e^{-\sqrt{\mu}t}.$$

# Функционал Ляпунова: сильно выпуклый случай

SC-AGF



## Упражнения: Heavy Ball и фазовое пространство



## Упражнение 1: Heavy Ball на квадратике

Пусть

$$f(x) = \frac{1}{2}(x - x^*)^\top A(x - x^*), \quad \mu I \preceq A \preceq LI, \quad \kappa = \frac{L}{\mu}.$$

Метод тяжёлого шарика Поляка:

$$x_{k+1} = x_k - \alpha \nabla f(x_k) + \beta(x_k - x_{k-1}).$$

## Упражнение 1: Heavy Ball на квадратике

Пусть

$$f(x) = \frac{1}{2}(x - x^*)^\top A(x - x^*), \quad \mu I \preceq A \preceq LI, \quad \kappa = \frac{L}{\mu}.$$

Метод тяжёлого шарика Поляка:

$$x_{k+1} = x_k - \alpha \nabla f(x_k) + \beta(x_k - x_{k-1}).$$

В собственных координатах  $A = Q\Lambda Q^\top$  каждая координата эволюционирует независимо:

$$\hat{x}_{k+1}^{(i)} = (1 - \alpha\lambda_i + \beta)\hat{x}_k^{(i)} - \beta\hat{x}_{k-1}^{(i)}.$$

То есть анализ сводится к семейству скалярных рекуррентных уравнений по  $\lambda_i \in [\mu, L]$ .

## Упражнение 1: параметры Поляка

Для сильно выпуклой квадратичной функции оптимизация худшего спектрального радиуса даёт:

$$\alpha_* = \frac{4}{(\sqrt{L} + \sqrt{\mu})^2}, \quad \beta_* = \left( \frac{\sqrt{L} - \sqrt{\mu}}{\sqrt{L} + \sqrt{\mu}} \right)^2.$$

## Упражнение 1: параметры Поляка

Для сильно выпуклой квадратичной функции оптимизация худшего спектрального радиуса даёт:

$$\alpha_* = \frac{4}{(\sqrt{L} + \sqrt{\mu})^2}, \quad \beta_* = \left( \frac{\sqrt{L} - \sqrt{\mu}}{\sqrt{L} + \sqrt{\mu}} \right)^2.$$

Обозначим

$$\rho_* = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}.$$

Тогда спектральный радиус матрицы итерации равен  $\rho_*$ , поэтому

$$\|z_k\| = \mathcal{O}(\rho_*^k), \quad f(x_k) - f^* = \mathcal{O}(\rho_*^{2k}), \quad z_k = \begin{bmatrix} \hat{x}_k \\ \hat{x}_{k-1} \end{bmatrix}.$$

## Упражнение 1: параметры Поляка

Для сильно выпуклой квадратичной функции оптимизация худшего спектрального радиуса даёт:

$$\alpha_* = \frac{4}{(\sqrt{L} + \sqrt{\mu})^2}, \quad \beta_* = \left( \frac{\sqrt{L} - \sqrt{\mu}}{\sqrt{L} + \sqrt{\mu}} \right)^2.$$

Обозначим

$$\rho_* = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}.$$

Тогда спектральный радиус матрицы итерации равен  $\rho_*$ , поэтому

$$\|z_k\| = \mathcal{O}(\rho_*^k), \quad f(x_k) - f^* = \mathcal{O}(\rho_*^{2k}), \quad z_k = \begin{bmatrix} \hat{x}_k \\ \hat{x}_{k-1} \end{bmatrix}.$$

Это квадратичная гарантия: она следует из спектрального анализа матрицы итерации, а не из общего анализа произвольной гладкой сильно выпуклой функции.

## Упражнение 1: строим непрерывный аналог

Перепишем НВ как уравнение на изменение шага:

$$x_{k+1} - 2x_k + x_{k-1} = -(1 - \beta)(x_k - x_{k-1}) - \alpha \nabla f(x_k).$$

## Упражнение 1: строим непрерывный аналог

Перепишем НВ как уравнение на изменение шага:

$$x_{k+1} - 2x_k + x_{k-1} = -(1 - \beta)(x_k - x_{k-1}) - \alpha \nabla f(x_k).$$

Положим  $\alpha = h^2$  и выберем семейство параметров

$$\beta = (1 - \sqrt{\mu} h)^2 = 1 - 2\sqrt{\mu} h + \mu h^2.$$

При

$$h_* = \frac{2}{\sqrt{L} + \sqrt{\mu}}$$

получаем ровно параметры Поляка:  $\alpha_* = h_*^2$ ,  $\beta_* = (1 - \sqrt{\mu} h_*)^2$ .

## Упражнение 1: предел при $h \rightarrow 0$

Делим уравнение НВ на  $h^2$ :

$$\frac{x_{k+1} - 2x_k + x_{k-1}}{h^2} + \frac{1 - \beta}{h} \frac{x_k - x_{k-1}}{h} + \nabla f(x_k) = 0.$$



## Упражнение 1: предел при $h \rightarrow 0$

Делим уравнение НВ на  $h^2$ :

$$\frac{x_{k+1} - 2x_k + x_{k-1}}{h^2} + \frac{1 - \beta}{h} \frac{x_k - x_{k-1}}{h} + \nabla f(x_k) = 0.$$

При  $t_k = kh$ ,  $x(t_k) = x_k$ :

$$\frac{x_{k+1} - 2x_k + x_{k-1}}{h^2} \rightarrow \ddot{x}(t), \quad \frac{x_k - x_{k-1}}{h} \rightarrow \dot{x}(t),$$

и

$$\frac{1 - \beta}{h} = 2\sqrt{\mu} - \mu h \rightarrow 2\sqrt{\mu}.$$

## Упражнение 1: предел при $h \rightarrow 0$

Делим уравнение НВ на  $h^2$ :

$$\frac{x_{k+1} - 2x_k + x_{k-1}}{h^2} + \frac{1 - \beta}{h} \frac{x_k - x_{k-1}}{h} + \nabla f(x_k) = 0.$$

При  $t_k = kh$ ,  $x(t_k) = x_k$ :

$$\frac{x_{k+1} - 2x_k + x_{k-1}}{h^2} \rightarrow \ddot{x}(t), \quad \frac{x_k - x_{k-1}}{h} \rightarrow \dot{x}(t),$$

и

$$\frac{1 - \beta}{h} = 2\sqrt{\mu} - \mu h \rightarrow 2\sqrt{\mu}.$$



$$\ddot{x}(t) + 2\sqrt{\mu} \dot{x}(t) + \nabla f(x(t)) = 0.$$

## Упражнение 1: первая анимация лекции

В первой анимации теперь сравниваются четыре траектории на одной и той же сильно выпуклой квадратичной функции:

- **GD:** дискретный градиентный спуск

## Упражнение 1: первая анимация лекции

В первой анимации теперь сравниваются четыре траектории на одной и той же сильно выпуклой квадратичной функции:

- **GD**: дискретный градиентный спуск
- **GF**: непрерывный градиентный поток

## Упражнение 1: первая анимация лекции

В первой анимации теперь сравниваются четыре траектории на одной и той же сильно выпуклой квадратичной функции:

- **GD**: дискретный градиентный спуск
- **GF**: непрерывный градиентный поток
- **NAG SC**: Нестеров для  $\mu$ -сильно выпуклого случая

## Упражнение 1: первая анимация лекции

В первой анимации теперь сравниваются четыре траектории на одной и той же сильно выпуклой квадратичной функции:

- **GD**: дискретный градиентный спуск
- **GF**: непрерывный градиентный поток
- **NAG SC**: Нестеров для  $\mu$ -сильно выпуклого случая
- **Polyak HB**: метод тяжёлого шарика с параметрами Поляка

## Упражнение 1: первая анимация лекции

В первой анимации теперь сравниваются четыре траектории на одной и той же сильно выпуклой квадратичной функции:

- **GD**: дискретный градиентный спуск
- **GF**: непрерывный градиентный поток
- **NAG SC**: Нестеров для  $\mu$ -сильно выпуклого случая
- **Polyak HB**: метод тяжёлого шарика с параметрами Поляка

## Упражнение 1: первая анимация лекции

В первой анимации теперь сравниваются четыре траектории на одной и той же сильно выпуклой квадратичной функции:

- **GD**: дискретный градиентный спуск
- **GF**: непрерывный градиентный поток
- **NAG SC**: Нестеров для  $\mu$ -сильно выпуклого случая
- **Polyak HB**: метод тяжёлого шарика с параметрами Поляка

Для NAG SC и Polyak HB в пределе возникает одно и то же ОДУ

$$\ddot{x} + 2\sqrt{\mu}\dot{x} + \nabla f(x) = 0,$$

но дискретные траектории отличаются: градиент берётся в разных точках и допустимые шаги задаются разными условиями.



## Упражнение 2: сведение к системе первого порядка

Возьмём ОДУ ускоренной динамики в сильно выпуклом случае:

$$\ddot{x}(t) + 2\sqrt{\mu} \dot{x}(t) + \nabla f(x(t)) = 0.$$

## Упражнение 2: сведение к системе первого порядка

Возьмём ОДУ ускоренной динамики в сильно выпуклом случае:

$$\ddot{x}(t) + 2\sqrt{\mu} \dot{x}(t) + \nabla f(x(t)) = 0.$$

Вводим скорость

$$v(t) = \dot{x}(t).$$

Тогда состояние становится вектором большей размерности:

$$z(t) = \begin{bmatrix} x(t) \\ v(t) \end{bmatrix} \in \mathbb{R}^{2d}.$$

## Упражнение 2: система первого порядка

Из  $v = \dot{x}$  получаем:

$$\dot{x} = v.$$

Из исходного ОДУ:

$$\dot{v} = \ddot{x} = -2\sqrt{\mu}v - \nabla f(x).$$

## Упражнение 2: система первого порядка

Из  $v = \dot{x}$  получаем:

$$\dot{x} = v.$$

Из исходного ОДУ:

$$\dot{v} = \ddot{x} = -2\sqrt{\mu}v - \nabla f(x).$$

Итого:  $\vdots$  {.callout-important appearance="simple"}

$$\frac{d}{dt} \begin{bmatrix} x \\ v \end{bmatrix} = \begin{bmatrix} v \\ -2\sqrt{\mu}v - \nabla f(x) \end{bmatrix}.$$

$\vdots$

Это уже ОДУ первого порядка, но в пространстве размерности  $2d$ .

## Упражнение 2: квадратичный случай

Если после сдвига минимума в ноль  $f(x) = \frac{1}{2}x^\top Ax$ , то  $\nabla f(x) = Ax$ , и система становится линейной:

$$\frac{d}{dt} \begin{bmatrix} x \\ v \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & I \\ -A & -2\sqrt{\mu} I \end{bmatrix}}_{M_c} \begin{bmatrix} x \\ v \end{bmatrix}.$$

## Упражнение 2: квадратичный случай

Если после сдвига минимума в ноль  $f(x) = \frac{1}{2}x^\top Ax$ , то  $\nabla f(x) = Ax$ , и система становится линейной:

$$\frac{d}{dt} \begin{bmatrix} x \\ v \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & I \\ -A & -2\sqrt{\mu} I \end{bmatrix}}_{M_c} \begin{bmatrix} x \\ v \end{bmatrix}.$$

После диагонализации  $A = Q\Lambda Q^\top$  снова получаем независимые двумерные блоки:

$$\frac{d}{dt} \begin{bmatrix} \hat{x}_i \\ \hat{v}_i \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\lambda_i & -2\sqrt{\mu} \end{bmatrix} \begin{bmatrix} \hat{x}_i \\ \hat{v}_i \end{bmatrix}.$$

## Упражнение 2: что видно из спектра

Для одного собственного направления характеристическое уравнение:

$$s^2 + 2\sqrt{\mu}s + \lambda_i = 0.$$

Его корни:

$$s_i^{\pm} = -\sqrt{\mu} \pm \sqrt{\mu - \lambda_i}.$$

## Упражнение 2: что видно из спектра

Для одного собственного направления характеристическое уравнение:

$$s^2 + 2\sqrt{\mu}s + \lambda_i = 0.$$

Его корни:

$$s_i^{\pm} = -\sqrt{\mu} \pm \sqrt{\mu - \lambda_i}.$$

Так как  $\lambda_i \geq \mu$ , вещественная часть всегда равна  $-\sqrt{\mu}$ :

$$\operatorname{Re} s_i^{\pm} = -\sqrt{\mu}.$$

Значит все моды затухают с огибающей  $e^{-\sqrt{\mu}t}$ ; направления с  $\lambda_i > \mu$  при этом могут осциллировать.



## Continuized Acceleration (Bach, 2021)

Проблема: дискретизация  $\alpha \rightarrow 0$  теряет информацию о методе. Можно ли построить непрерывный аналог, **различающий** алгоритмы?

---

<sup>2</sup>Francis Bach: Continuized Acceleration

## Continuized Acceleration (Bach, 2021)

Проблема: дискретизация  $\alpha \rightarrow 0$  теряет информацию о методе. Можно ли построить непрерывный аналог, **различающий** алгоритмы?

**Идея**<sup>2</sup>: градиентные шаги в **случайные** моменты времени (пуассоновский процесс интенсивности  $1/\gamma$ ):

---

<sup>2</sup>Francis Bach: Continuized Acceleration

## Continuized Acceleration (Bach, 2021)

Проблема: дискретизация  $\alpha \rightarrow 0$  теряет информацию о методе. Можно ли построить непрерывный аналог, **различающий** алгоритмы?

**Идея**<sup>2</sup>: градиентные шаги в **случайные** моменты времени (пуассоновский процесс интенсивности  $1/\gamma$ ):

Между прыжками — непрерывная “смешивающая” динамика:

$$dx_t = \sqrt{\gamma\mu}(z_t - x_t) dt, \quad dz_t = \sqrt{\gamma\mu}(x_t - z_t) dt$$

В моменты прыжков  $T_k$ :

$$x_{T_k} \leftarrow \gamma \nabla f(x_{T_k^-})$$

---

<sup>2</sup>Francis Bach: Continuized Acceleration

# Continuized Acceleration (Bach, 2021)

Проблема: дискретизация  $\alpha \rightarrow 0$  теряет информацию о методе. Можно ли построить непрерывный аналог, **различающий** алгоритмы?

**Идея**<sup>2</sup>: градиентные шаги в **случайные** моменты времени (пуассоновский процесс интенсивности  $1/\gamma$ ):

Между прыжками — непрерывная “смешивающая” динамика:

$$dx_t = \sqrt{\gamma\mu}(z_t - x_t) dt, \quad dz_t = \sqrt{\gamma\mu}(x_t - z_t) dt$$

В моменты прыжков  $T_k$ :

$$x_{T_k} \leftarrow \gamma \nabla f(x_{T_k^-})$$

**! Гарантия сходимости**

$$\mathbb{E}[f(x_t) - f^*] \leq C e^{-\sqrt{\gamma\mu}t}$$

---

<sup>2</sup>Francis Bach: Continuized Acceleration

# Continuized Acceleration (Bach, 2021)

Проблема: дискретизация  $\alpha \rightarrow 0$  теряет информацию о методе. Можно ли построить непрерывный аналог, **различающий** алгоритмы?

**Идея**<sup>2</sup>: градиентные шаги в **случайные** моменты времени (пуассоновский процесс интенсивности  $1/\gamma$ ):

Между прыжками — непрерывная “смешивающая” динамика:

$$dx_t = \sqrt{\gamma\mu}(z_t - x_t) dt, \quad dz_t = \sqrt{\gamma\mu}(x_t - z_t) dt$$

В моменты прыжков  $T_k$ :

$$x_{T_k} = \gamma \nabla f(x_{T_k^-})$$

**Преимущества:** явно различает NAG и Heavy Ball; работает в асинхронных настройках; мост между дискретным и непрерывным временем.

**! Гарантия сходимости**

$$\mathbb{E}[f(x_t) - f^*] \leq C e^{-\sqrt{\gamma\mu}t}$$

---

<sup>2</sup>Francis Bach: Continuized Acceleration

Свежие результаты: 2025–2026

# Поточечная сходимость NAG: 40 лет спустя

**Открытая проблема с 1983 года:** сходятся ли сами итераты NAG, а не только значения функции?

---

<sup>3</sup>Point Convergence of Nesterov's Accelerated Gradient Method: A ChatGPT-Assisted Proof, arXiv:2510.23513

## Поточечная сходимость NAG: 40 лет спустя

**Открытая проблема с 1983 года:** сходятся ли сами итераты NAG, а не только значения функции?

**Теорема Jang–Ryu (2025)**<sup>3</sup>: для выпуклой  $L$ -гладкой  $f$  и NAG с шагом  $1/L$ :

$$x_k, y_k \longrightarrow x_\infty \in \arg \min f.$$

---

<sup>3</sup>Point Convergence of Nesterov's Accelerated Gradient Method: A ChatGPT-Assisted Proof, arXiv:2510.23513



# Поточечная сходимость NAG: 40 лет спустя

**Открытая проблема с 1983 года:** сходятся ли сами итераты NAG, а не только значения функции?

**Теорема Jang–Ryu (2025)**<sup>3</sup>: для выпуклой  $L$ -гладкой  $f$  и NAG с шагом  $1/L$ :

$$x_k, y_k \longrightarrow x_\infty \in \arg \min f.$$

**Важно:**  $\mathcal{O}(1/k^2)$  для  $f(x_k) - f^*$  давно известно. Новый результат сильнее: последовательность не просто улучшает значение, а выбирает конкретную точку минимума.

---

<sup>3</sup>Point Convergence of Nesterov's Accelerated Gradient Method: A ChatGPT-Assisted Proof, arXiv:2510.23513

# Поточечная сходимость NAG: 40 лет спустя

**Открытая проблема с 1983 года:** сходятся ли сами итераты NAG, а не только значения функции?

**Теорема Jang–Ryu (2025)**<sup>3</sup>: для выпуклой  $L$ -гладкой  $f$  и NAG с шагом  $1/L$ :

$$x_k, y_k \longrightarrow x_\infty \in \arg \min f.$$

**Важно:**  $\mathcal{O}(1/k^2)$  для  $f(x_k) - f^*$  давно известно. Новый результат сильнее: последовательность не просто улучшает значение, а выбирает конкретную точку минимума.

## **i** Роль AI в доказательстве

Авторы явно пишут, что доказательство было heavily ChatGPT-assisted. Смысловой ход: сначала доказывается сходимость непрерывного ОДУ, затем дискретный случай через тонкий Lyapunov/Toeplitz аргумент и контроль хвостов.

<sup>3</sup>Point Convergence of Nesterov's Accelerated Gradient Method: A ChatGPT-Assisted Proof, arXiv:2510.23513

# Нестеровский поток и бесконечная длина пути

Ryu (2026)<sup>4</sup> — доказательство получено внутренней моделью OpenAI.

## ! Теорема

Существует выпуклая  $C^1$ -функция  $f: \mathbb{R}^2 \rightarrow \mathbb{R}$  такая, что поток Нестерова  $x(t)$  сходится к минимуму  $x^*$ , но  $\int_0^\infty \|\dot{x}(t)\| dt = +\infty$ .

---

<sup>4</sup>Nesterov Flow May Travel Infinitely Long..., arXiv:2604.06651

# Нестеровский поток и бесконечная длина пути

Ryu (2026)<sup>4</sup> — доказательство получено внутренней моделью OpenAI.

## ! Теорема

Существует выпуклая  $C^1$ -функция  $f: \mathbb{R}^2 \rightarrow \mathbb{R}$  такая, что поток Нестерова  $x(t)$  сходится к минимуму  $x^*$ , но  $\int_0^\infty \|\dot{x}(t)\| dt = +\infty$ .

**Идеи конструкции:**  $f_{\varepsilon,a}(x) = F(\|x\|) + \varepsilon \frac{1}{2}(x_1 - a)_+^2$ , где  $F'(r) = 1/(-\log r)$  около нуля; односторонний bump создаёт ненулевой angular momentum; в радиальном ядре сохраняется  $t^3 \det(x(t), \dot{x}(t)) = \text{const}$ , откуда  $\|\dot{x}_\perp(t)\| \gtrsim 1/(t \log t)$  и  $\int_T^\infty dt/(t \log t) = +\infty$ .

<sup>4</sup>Nesterov Flow May Travel Infinitely Long..., arXiv:2604.06651

# Нестеровский поток и бесконечная длина пути

Ryu (2026)<sup>4</sup> — доказательство получено внутренней моделью OpenAI.

## ! Теорема

Существует выпуклая  $C^1$ -функция  $f: \mathbb{R}^2 \rightarrow \mathbb{R}$  такая, что поток Нестерова  $x(t)$  сходится к минимуму  $x^*$ , но  $\int_0^\infty \|\dot{x}(t)\| dt = +\infty$ .

**Идеи конструкции:**  $f_{\varepsilon,a}(x) = F(\|x\|) + \varepsilon \frac{1}{2}(x_1 - a)_+^2$ , где  $F'(r) = 1/(-\log r)$  около нуля; односторонний bump создаёт ненулевой angular momentum; в радиальном ядре сохраняется  $t^3 \det(x(t), \dot{x}(t)) = \text{const}$ , откуда  $\|\dot{x}_\perp(t)\| \gtrsim 1/(t \log t)$  и  $\int_T^\infty dt/(t \log t) = +\infty$ .

**Вывод:** несмотря на сходимость  $f(x(t)) \rightarrow f^*$ , траектория бесконечно **спиралит** вокруг минимума.

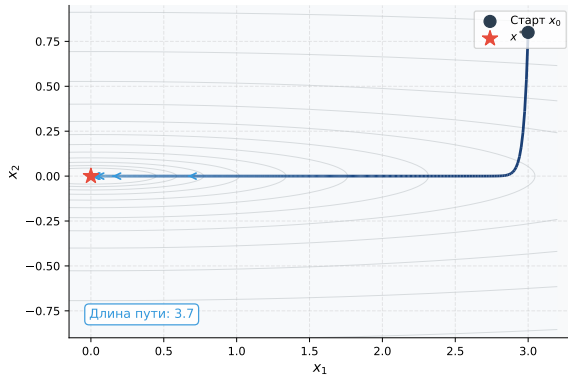
<sup>4</sup>Nesterov Flow May Travel Infinitely Long..., arXiv:2604.06651

# Визуализация спирали Нестерова

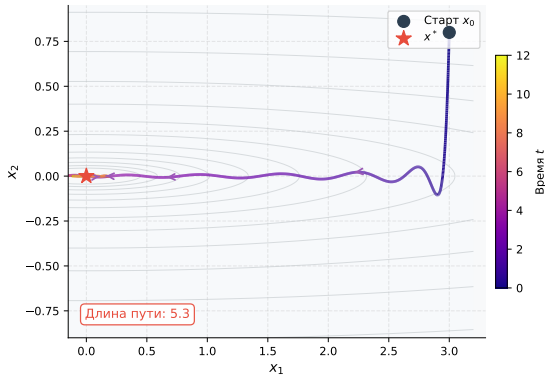
Нестеровский поток и бесконечная длина пути:  $f(x) = (x_1^2 + 100x_2^2)/2$ ,  $\kappa = 100$

GF монотонен; AGF осциллирует — длина пути растёт — при кратном росте кривизны длина стремится к бесконечности

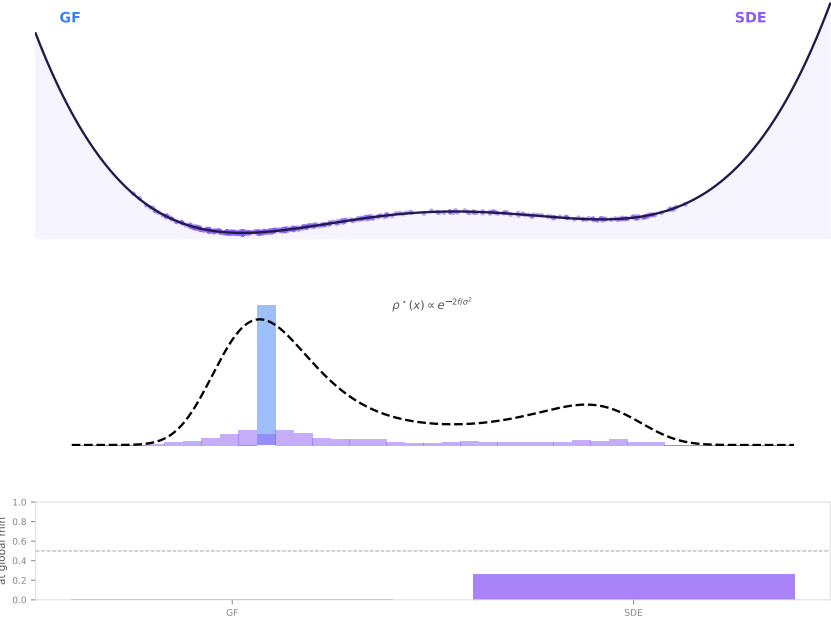
GF — монотонный спуск



AGF (Нестеров) — зигзаг,  $\kappa=100$



# SDE vs GF: побег из локального минимума



## Стохастический градиентный поток и уравнение Фоккера–Планка



# Стохастический градиентный поток

Как моделировать стохастичность в непрерывном времени?

# Стохастический градиентный поток

Как моделировать стохастичность в непрерывном времени?

Добавим случайный шум к градиентному потоку:  $\frac{dx}{dt} = -\nabla f(x) + \xi$ , где  $\xi \sim \mathcal{N}(0, \sigma^2 I)$ .

# Стохастический градиентный поток

Как моделировать стохастичность в непрерывном времени?

Добавим случайный шум к градиентному потоку:  $\frac{dx}{dt} = -\nabla f(x) + \xi$ , где  $\xi \sim \mathcal{N}(0, \sigma^2 I)$ .

Запишем **стохастическое дифференциальное уравнение (СДУ)**:



$$dx(t) = -\nabla f(x(t)) dt + \sigma dW(t)$$

# Стохастический градиентный поток

Как моделировать стохастичность в непрерывном времени?

Добавим случайный шум к градиентному потоку:  $\frac{dx}{dt} = -\nabla f(x) + \xi$ , где  $\xi \sim \mathcal{N}(0, \sigma^2 I)$ .

Запишем **стохастическое дифференциальное уравнение (СДУ)**:



$$dx(t) = -\nabla f(x(t)) dt + \sigma dW(t)$$

Здесь  $W(t)$  — **процесс Винера** (броуновское движение). Два равноправных взгляда на анализ:

**(а) Траектории:** следим за отдельными реализациями  $x(t)$  **(b) Плотность:** следим за эволюцией  $\rho(x, t)$  — плотностью распределения частиц

# Уравнение Фоккера–Планка

Плотность вероятности  $\rho(x, t)$  удовлетворяет уравнению в частных производных:

**! Уравнение Фоккера–Планка**

$$\frac{\partial \rho}{\partial t} = \nabla \cdot (\rho \nabla f) + \frac{\sigma^2}{2} \Delta \rho$$

# Уравнение Фоккера–Планка

Плотность вероятности  $\rho(x, t)$  удовлетворяет уравнению в частных производных:

## ! Уравнение Фоккера–Планка

$$\frac{\partial \rho}{\partial t} = \nabla \cdot (\rho \nabla f) + \frac{\sigma^2}{2} \Delta \rho$$

Интерпретация двух слагаемых:

- $\nabla \cdot (\rho \nabla f)$  — **дрейфовый член**: детерминированный градиентный поток переносит массу к минимуму  $f$

# Уравнение Фоккера–Планка

Плотность вероятности  $\rho(x, t)$  удовлетворяет уравнению в частных производных:

## ! Уравнение Фоккера–Планка

$$\frac{\partial \rho}{\partial t} = \nabla \cdot (\rho \nabla f) + \frac{\sigma^2}{2} \Delta \rho$$

Интерпретация двух слагаемых:

- $\nabla \cdot (\rho \nabla f)$  — **дрейфовый член**: детерминированный градиентный поток переносит массу к минимуму  $f$
- $\frac{\sigma^2}{2} \Delta \rho$  — **диффузионный член**: шум Винера размывает плотность

# Уравнение Фоккера–Планка

Плотность вероятности  $\rho(x, t)$  удовлетворяет уравнению в частных производных:

## ! Уравнение Фоккера–Планка

$$\frac{\partial \rho}{\partial t} = \nabla \cdot (\rho \nabla f) + \frac{\sigma^2}{2} \Delta \rho$$

Интерпретация двух слагаемых:

- $\nabla \cdot (\rho \nabla f)$  — **дрейфовый член**: детерминированный градиентный поток переносит массу к минимуму  $f$
- $\frac{\sigma^2}{2} \Delta \rho$  — **диффузионный член**: шум Винера размывает плотность



# Уравнение Фоккера–Планка

Плотность вероятности  $\rho(x, t)$  удовлетворяет уравнению в частных производных:

## ! Уравнение Фоккера–Планка

$$\frac{\partial \rho}{\partial t} = \nabla \cdot (\rho \nabla f) + \frac{\sigma^2}{2} \Delta \rho$$

Интерпретация двух слагаемых:

- $\nabla \cdot (\rho \nabla f)$  — **дрейфовый член**: детерминированный градиентный поток переносит массу к минимуму  $f$
- $\frac{\sigma^2}{2} \Delta \rho$  — **диффузионный член**: шум Винера размывает плотность

Стационарное распределение ( $\partial \rho / \partial t = 0$ ):

$$\rho^*(x) \propto \exp\left(-\frac{2f(x)}{\sigma^2}\right)$$

Это **распределение Гиббса** (больцмановское распределение)!

# Фоккер–Планк как градиентный поток в пространстве Вассерштейна

Уравнение Фоккера–Планка — это не просто ОДУ для распределений. Оно обладает вариационной структурой!

## Фоккер–Планк как градиентный поток в пространстве Вассерштейна

Уравнение Фоккера–Планка — это не просто ОДУ для распределений. Оно обладает вариационной структурой!

**Функционал свободной энергии Гельмгольца:**

$$\mathcal{F}(\rho) = \underbrace{\int \rho(x) f(x) dx}_{\text{потенциальная энергия}} + \underbrace{\frac{\sigma^2}{2} \int \rho(x) \log \rho(x) dx}_{\text{энтропия (отрицательная)}}$$

## Фоккер–Планк как градиентный поток в пространстве Вассерштейна

Уравнение Фоккера–Планка — это не просто ОДУ для распределений. Оно обладает вариационной структурой!

Функционал свободной энергии Гельмгольца:

$$\mathcal{F}(\rho) = \underbrace{\int \rho(x) f(x) dx}_{\text{потенциальная энергия}} + \underbrace{\frac{\sigma^2}{2} \int \rho(x) \log \rho(x) dx}_{\text{энтропия (отрицательная)}}$$

### ! Теорема (Jordan–Kinderlehrer–Otto, 1998)

Уравнение Фоккера–Планка — это **градиентный поток** функционала  $\mathcal{F}$  в метрике Вассерштейна  $\mathcal{W}_2$ :

$$\frac{\partial \rho}{\partial t} = -\nabla_{\mathcal{W}_2} \mathcal{F}(\rho)$$

## Фоккер–Планк как градиентный поток в пространстве Вассерштейна

Уравнение Фоккера–Планка — это не просто ОДУ для распределений. Оно обладает вариационной структурой!

**Функционал свободной энергии Гельмгольца:**

$$\mathcal{F}(\rho) = \underbrace{\int \rho(x) f(x) dx}_{\text{потенциальная энергия}} + \underbrace{\frac{\sigma^2}{2} \int \rho(x) \log \rho(x) dx}_{\text{энтропия (отрицательная)}}$$

### ! Теорема (Jordan–Kinderlehrer–Otto, 1998)

Уравнение Фоккера–Планка — это **градиентный поток** функционала  $\mathcal{F}$  в метрике Вассерштейна  $\mathcal{W}_2$ :

$$\frac{\partial \rho}{\partial t} = -\nabla_{\mathcal{W}_2} \mathcal{F}(\rho)$$

**Схема ЖКО — проксимальный метод для распределений:**

$$\rho_{k+1} = \arg \min_{\rho} \left[ \mathcal{F}(\rho) + \frac{1}{2\alpha} \mathcal{W}_2^2(\rho, \rho_k) \right]$$

Это точный аналог проксимального метода, но в **пространстве мер**! Каждый шаг ЖКО = одна итерация “приближённого” броуновского движения + дрейф.

## Интерпретация стационарного распределения

Стационарное распределение  $\rho^*(x) \propto \exp\left(-\frac{2f(x)}{\sigma^2}\right)$  имеет богатую интерпретацию:

# Интерпретация стационарного распределения

Стационарное распределение  $\rho^*(x) \propto \exp\left(-\frac{2f(x)}{\sigma^2}\right)$  имеет богатую интерпретацию:

**Влияние параметра шума  $\sigma$ :**

- При  $\sigma \rightarrow 0$ : распределение **концентрируется** в глобальном минимуме  $x^*$

# Интерпретация стационарного распределения

Стационарное распределение  $\rho^*(x) \propto \exp\left(-\frac{2f(x)}{\sigma^2}\right)$  имеет богатую интерпретацию:

**Влияние параметра шума  $\sigma$ :**

- При  $\sigma \rightarrow 0$ : распределение **концентрируется** в глобальном минимуме  $x^*$
- Детерминированный предел:  $\rho^* \rightarrow \delta(x - x^*)$



# Интерпретация стационарного распределения

Стационарное распределение  $\rho^*(x) \propto \exp\left(-\frac{2f(x)}{\sigma^2}\right)$  имеет богатую интерпретацию:

**Влияние параметра шума  $\sigma$ :**

- При  $\sigma \rightarrow 0$ : распределение **концентрируется** в глобальном минимуме  $x^*$
- Детерминированный предел:  $\rho^* \rightarrow \delta(x - x^*)$
- При  $\sigma \rightarrow \infty$ : распределение **размывается** по всему пространству

# Интерпретация стационарного распределения

Стационарное распределение  $\rho^*(x) \propto \exp\left(-\frac{2f(x)}{\sigma^2}\right)$  имеет богатую интерпретацию:

**Влияние параметра шума  $\sigma$ :**

- При  $\sigma \rightarrow 0$ : распределение **концентрируется** в глобальном минимуме  $x^*$
- Детерминированный предел:  $\rho^* \rightarrow \delta(x - x^*)$
- При  $\sigma \rightarrow \infty$ : распределение **размывается** по всему пространству
- «Случайный» предел:  $\rho^* \rightarrow \text{const}$  (равномерное)

# Интерпретация стационарного распределения

Стационарное распределение  $\rho^*(x) \propto \exp\left(-\frac{2f(x)}{\sigma^2}\right)$  имеет богатую интерпретацию:

**Влияние параметра шума  $\sigma$ :**

- При  $\sigma \rightarrow 0$ : распределение **концентрируется** в глобальном минимуме  $x^*$
- Детерминированный предел:  $\rho^* \rightarrow \delta(x - x^*)$
- При  $\sigma \rightarrow \infty$ : распределение **размывается** по всему пространству
- «Случайный» предел:  $\rho^* \rightarrow \text{const}$  (равномерное)

# Интерпретация стационарного распределения

Стационарное распределение  $\rho^*(x) \propto \exp\left(-\frac{2f(x)}{\sigma^2}\right)$  имеет богатую интерпретацию:

**Влияние параметра шума  $\sigma$ :**

- При  $\sigma \rightarrow 0$ : распределение **концентрируется** в глобальном минимуме  $x^*$
- Детерминированный предел:  $\rho^* \rightarrow \delta(x - x^*)$
- При  $\sigma \rightarrow \infty$ : распределение **размывается** по всему пространству
- «Случайный» предел:  $\rho^* \rightarrow \text{const}$  (равномерное)

**Связь с термодинамикой:** температура  $T \propto \sigma^2$  / аналогия с отжигом.

# Интерпретация стационарного распределения

Стационарное распределение  $\rho^*(x) \propto \exp\left(-\frac{2f(x)}{\sigma^2}\right)$  имеет богатую интерпретацию:

**Влияние параметра шума  $\sigma$ :**

- При  $\sigma \rightarrow 0$ : распределение **концентрируется** в глобальном минимуме  $x^*$
- Детерминированный предел:  $\rho^* \rightarrow \delta(x - x^*)$
- При  $\sigma \rightarrow \infty$ : распределение **размывается** по всему пространству
- «Случайный» предел:  $\rho^* \rightarrow \text{const}$  (равномерное)

**Связь с термодинамикой:** температура  $T \propto \sigma^2$  / аналогия с отжигом.

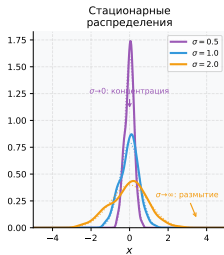
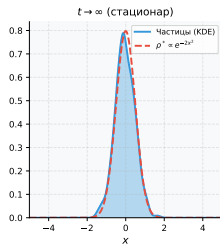
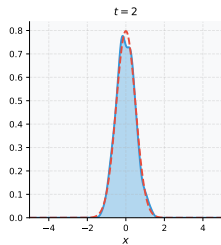
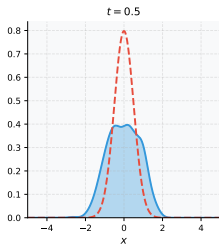
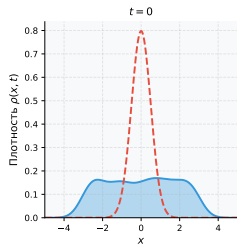
**Связь со схемой JKO (Jordan–Kinderlehrer–Otto):** уравнение Фоккера–Планка — это градиентный поток функционала свободной энергии Гельмгольца

$$\mathcal{F}(\rho) = \int \rho(x) f(x) dx + \frac{\sigma^2}{2} \int \rho(x) \log \rho(x) dx$$

в пространстве Вассерштейна  $\mathcal{W}_2$  — JKO-дискретизация этого потока даёт алгоритмы семплирования.

# Эволюция плотности: метод частиц

Уравнение Фоккера–Планка:  $dx = -2x dt + \sigma dW$   
 $N = 500$  частиц



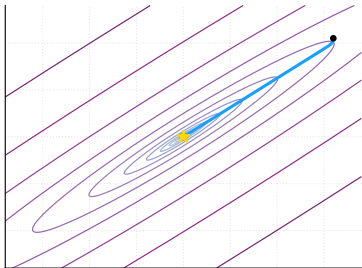
## Эксперименты

# GF / AGF / GD

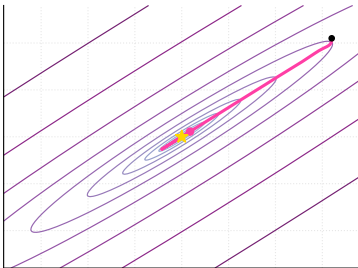
GF / AGF / GD

quadratic  $\kappa = 80$

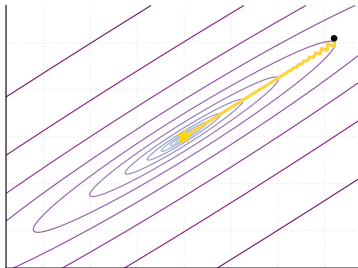
GF



AGF

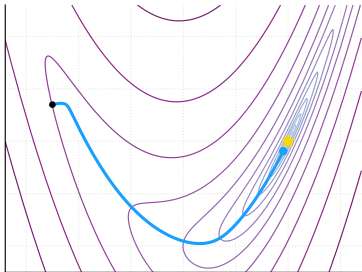


GD

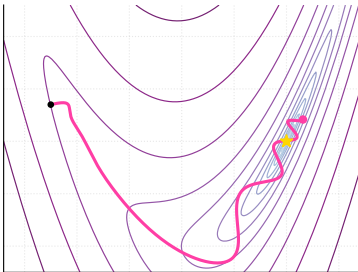


Rosenbrock

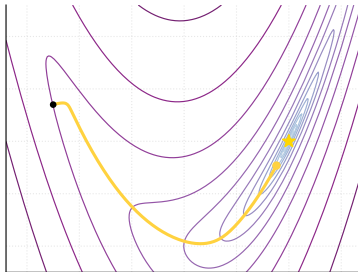
GF



AGF



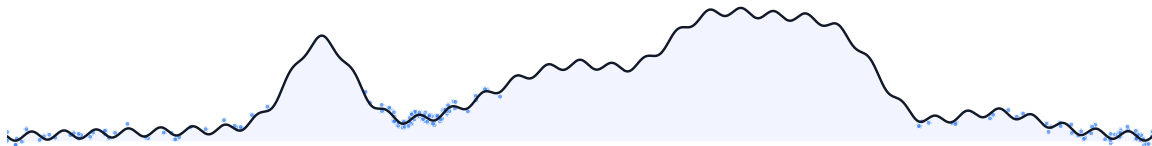
GD



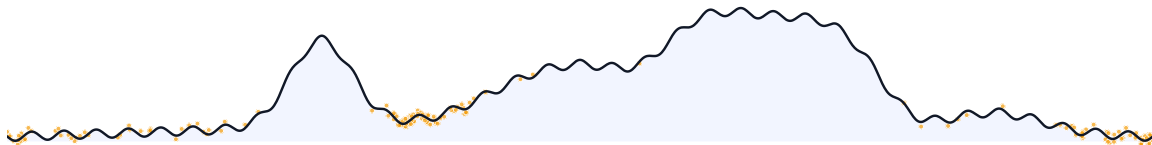


# Scalar dynamics: три механики на одной функции

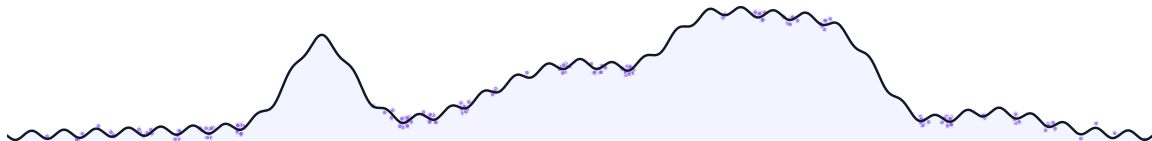
GF



HB



NAG



# Сравнение траекторий GF, AGF и GD

Градиентный поток, ускоренный GF и дискретный GD

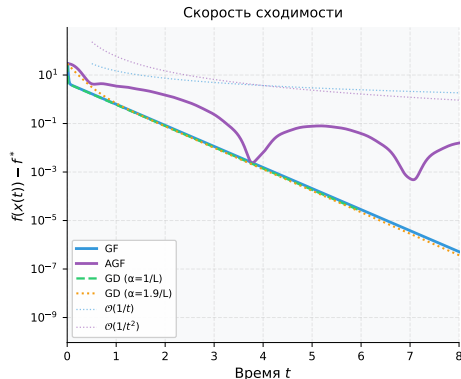
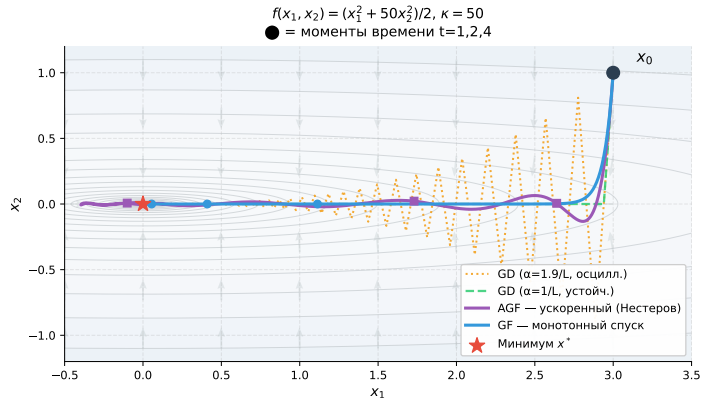
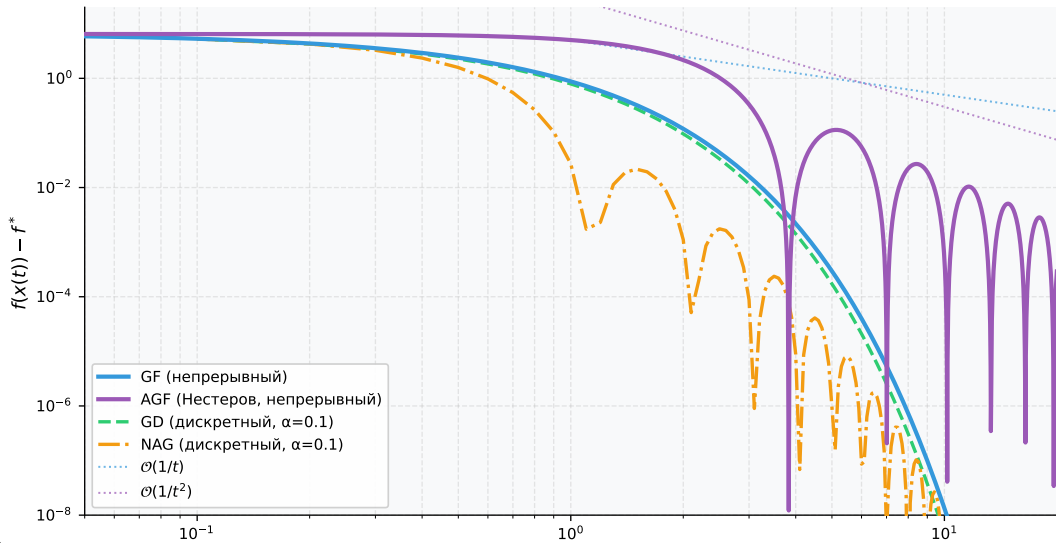


Figure 12: Слева — траектории в пространстве на  $f=(x_1^2+50x_2^2)/2$  ( $\kappa=50$ ): GF монотонно следует антиградиенту, AGF ускоряется с инерцией, GD с шагом  $\alpha=1.9/L$  осциллирует. Справа — скорости сходимости: AGF даёт  $\mathcal{O}(1/t^2)$  против  $\mathcal{O}(1/t)$  у GF.

# Скорости сходимости: теория vs эксперимент

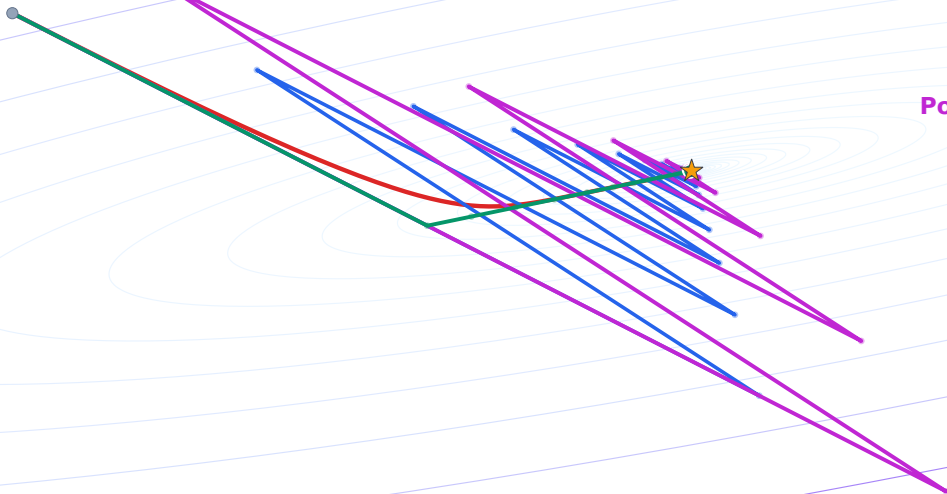
Скорости сходимости: GF, AGF и дискретные аналоги

$$f(x_1, x_2) = \frac{1}{2}(x_1^2 + x_2^2)$$



GD / GF / NAG SC / Polyak HB

GD  
GF  
NAG SC  
Polyak HB



## Central Flows (ICLR 2025): edge of stability

**Cohen et al. (2025)**<sup>5</sup>: моделируют не каждую итерацию, а усреднённую траекторию GD/RMSProp.

---

<sup>5</sup>Understanding Optimization in Deep Learning with Central Flows, arXiv:2410.24206

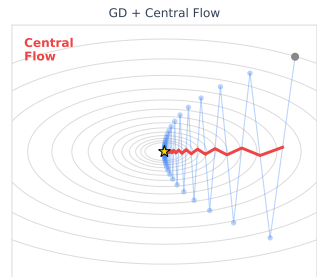
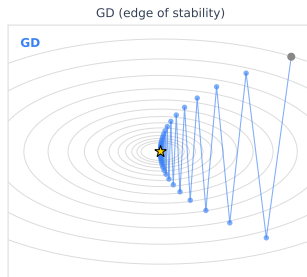
# Central Flows (ICLR 2025): edge of stability

**Cohen et al. (2025)**<sup>5</sup>: моделируют не каждую итерацию, а усреднённую траекторию GD/RMSProp.

В режиме edge of stability шаги прыгают вокруг центра:

$$w_t = \bar{w}_t + \delta_t, \quad w_{t+1} = \bar{w}_t - \delta_t.$$

Central Flow — ОДУ для  $\bar{w}_t$ , которое предсказывает медленную drift-динамику там, где gradient flow уже слеп.



@fminxyz

<sup>5</sup>Understanding Optimization in Deep Learning with Central Flows, arXiv:2410.24206

## Rod Flow (2026): точка стала стержнем

**Regis–Chewi (2026)<sup>6</sup>:** вместо одной точки  
держим центр и extent:

$$\bar{w}_t = \frac{w_t + w_{t+1}}{2}, \quad \Sigma_t = (w_{t+1} - \bar{w}_t)(w_{t+1} - \bar{w}_t)^\top.$$

---

<sup>6</sup>Rod Flow: A Continuous-Time Model for Gradient Descent at the Edge of Stability, arXiv:2602.01480

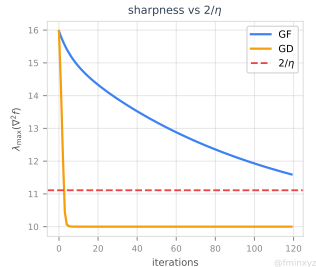
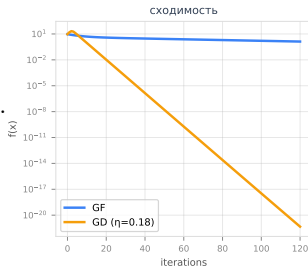
# Rod Flow (2026): точка стала стержнем

**Regis–Chewi (2026)<sup>6</sup>:** вместо одной точки держим центр и extent:

$$\bar{w}_t = \frac{w_t + w_{t+1}}{2}, \quad \Sigma_t = (w_{t+1} - \bar{w}_t)(w_{t+1} - \bar{w}_t)^\top.$$

Модель объясняет self-stabilization: sharpness зажимается около критического порога

$$\lambda_{\max}(\nabla^2 f) \approx \frac{2}{\eta}.$$



<sup>6</sup>Rod Flow: A Continuous-Time Model for Gradient Descent at the Edge of Stability, arXiv:2602.01480



## Источники

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2016)

## Источники

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2016)
- Andersen Ang — Convergence of Gradient Flow

## Источники

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2016)
- Andersen Ang — Convergence of Gradient Flow
- Francis Bach — Gradient flows (blog)

## Источники

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2016)
- Andersen Ang — Convergence of Gradient Flow
- Francis Bach — Gradient flows (blog)
- Francis Bach (2021) — Continuized Acceleration

## Источники

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2016)
- Andersen Ang — Convergence of Gradient Flow
- Francis Bach — Gradient flows (blog)
- Francis Bach (2021) — Continuized Acceleration
- Off Convex Path — Gradient flow и gradient descent

## Источники

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2016)
- Andersen Ang — Convergence of Gradient Flow
- Francis Bach — Gradient flows (blog)
- Francis Bach (2021) — Continuized Acceleration
- Off Convex Path — Gradient flow и gradient descent
- Polyak (1964) — Some methods of speeding up the convergence of iteration methods

## Источники

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2016)
- Andersen Ang — Convergence of Gradient Flow
- Francis Bach — Gradient flows (blog)
- Francis Bach (2021) — Continuized Acceleration
- Off Convex Path — Gradient flow и gradient descent
- Polyak (1964) — Some methods of speeding up the convergence of iteration methods
- Heavy Ball Method — fmin.xyz

## Источники

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2016)
- Andersen Ang — Convergence of Gradient Flow
- Francis Bach — Gradient flows (blog)
- Francis Bach (2021) — Continuized Acceleration
- Off Convex Path — Gradient flow и gradient descent
- Polyak (1964) — Some methods of speeding up the convergence of iteration methods
- Heavy Ball Method — fmin.xyz
- Jang & Ryu (2025) — Point Convergence of Nesterov's Accelerated Gradient Method



- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2016)
- Andersen Ang — Convergence of Gradient Flow
- Francis Bach — Gradient flows (blog)
- Francis Bach (2021) — Continuized Acceleration
- Off Convex Path — Gradient flow и gradient descent
- Polyak (1964) — Some methods of speeding up the convergence of iteration methods
- Heavy Ball Method — fmin.xyz
- Jang & Ryu (2025) — Point Convergence of Nesterov's Accelerated Gradient Method
- Ryu (2026) — Nesterov Flow May Travel Infinitely Long to Converge to a Minimizer

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2016)
- Andersen Ang — Convergence of Gradient Flow
- Francis Bach — Gradient flows (blog)
- Francis Bach (2021) — Continuized Acceleration
- Off Convex Path — Gradient flow и gradient descent
- Polyak (1964) — Some methods of speeding up the convergence of iteration methods
- Heavy Ball Method — fmin.xyz
- Jang & Ryu (2025) — Point Convergence of Nesterov's Accelerated Gradient Method
- Ryu (2026) — Nesterov Flow May Travel Infinitely Long to Converge to a Minimizer
- Cohen et al. (2025) — Understanding Optimization in Deep Learning with Central Flows

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2016)
- Andersen Ang — Convergence of Gradient Flow
- Francis Bach — Gradient flows (blog)
- Francis Bach (2021) — Continuized Acceleration
- Off Convex Path — Gradient flow и gradient descent
- Polyak (1964) — Some methods of speeding up the convergence of iteration methods
- Heavy Ball Method — fmin.xyz
- Jang & Ryu (2025) — Point Convergence of Nesterov's Accelerated Gradient Method
- Ryu (2026) — Nesterov Flow May Travel Infinitely Long to Converge to a Minimizer
- Cohen et al. (2025) — Understanding Optimization in Deep Learning with Central Flows
- Regis & Chewi (2026) — Rod Flow: A Continuous-Time Model for GD at the Edge of Stability

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2016)
- Andersen Ang — Convergence of Gradient Flow
- Francis Bach — Gradient flows (blog)
- Francis Bach (2021) — Continuized Acceleration
- Off Convex Path — Gradient flow и gradient descent
- Polyak (1964) — Some methods of speeding up the convergence of iteration methods
- Heavy Ball Method — fmin.xyz
- Jang & Ryu (2025) — Point Convergence of Nesterov's Accelerated Gradient Method
- Ryu (2026) — Nesterov Flow May Travel Infinitely Long to Converge to a Minimizer
- Cohen et al. (2025) — Understanding Optimization in Deep Learning with Central Flows
- Regis & Chewi (2026) — Rod Flow: A Continuous-Time Model for GD at the Edge of Stability
- Stochastic gradient algorithms from ODE splitting perspective

## Источники

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2016)
- Andersen Ang — Convergence of Gradient Flow
- Francis Bach — Gradient flows (blog)
- Francis Bach (2021) — Continuized Acceleration
- Off Convex Path — Gradient flow и gradient descent
- Polyak (1964) — Some methods of speeding up the convergence of iteration methods
- Heavy Ball Method — fmin.xyz
- Jang & Ryu (2025) — Point Convergence of Nesterov's Accelerated Gradient Method
- Ryu (2026) — Nesterov Flow May Travel Infinitely Long to Converge to a Minimizer
- Cohen et al. (2025) — Understanding Optimization in Deep Learning with Central Flows
- Regis & Chewi (2026) — Rod Flow: A Continuous-Time Model for GD at the Edge of Stability
- Stochastic gradient algorithms from ODE splitting perspective
- Introduction to Gradient Flows in the 2-Wasserstein Space

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2016)
- Andersen Ang — Convergence of Gradient Flow
- Francis Bach — Gradient flows (blog)
- Francis Bach (2021) — Continuized Acceleration
- Off Convex Path — Gradient flow и gradient descent
- Polyak (1964) — Some methods of speeding up the convergence of iteration methods
- Heavy Ball Method — fmin.xyz
- Jang & Ryu (2025) — Point Convergence of Nesterov's Accelerated Gradient Method
- Ryu (2026) — Nesterov Flow May Travel Infinitely Long to Converge to a Minimizer
- Cohen et al. (2025) — Understanding Optimization in Deep Learning with Central Flows
- Regis & Chewi (2026) — Rod Flow: A Continuous-Time Model for GD at the Edge of Stability
- Stochastic gradient algorithms from ODE splitting perspective
- Introduction to Gradient Flows in the 2-Wasserstein Space
- Jordan, Kinderlehrer, Otto (1998) — The Variational Formulation of the Fokker–Planck Equation

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2016)
- Andersen Ang — Convergence of Gradient Flow
- Francis Bach — Gradient flows (blog)
- Francis Bach (2021) — Continuized Acceleration
- Off Convex Path — Gradient flow и gradient descent
- Polyak (1964) — Some methods of speeding up the convergence of iteration methods
- Heavy Ball Method — fmin.xyz
- Jang & Ryu (2025) — Point Convergence of Nesterov's Accelerated Gradient Method
- Ryu (2026) — Nesterov Flow May Travel Infinitely Long to Converge to a Minimizer
- Cohen et al. (2025) — Understanding Optimization in Deep Learning with Central Flows
- Regis & Chewi (2026) — Rod Flow: A Continuous-Time Model for GD at the Edge of Stability
- Stochastic gradient algorithms from ODE splitting perspective
- Introduction to Gradient Flows in the 2-Wasserstein Space
- Jordan, Kinderlehrer, Otto (1998) — The Variational Formulation of the Fokker–Planck Equation
- OSQP: Stellato, Banjac, Goulart, Bemporad, Boyd, *Math. Prog. Comp.* 12(4), 2020 — osqp.org

## Источники

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2016)
- Andersen Ang — Convergence of Gradient Flow
- Francis Bach — Gradient flows (blog)
- Francis Bach (2021) — Continuized Acceleration
- Off Convex Path — Gradient flow и gradient descent
- Polyak (1964) — Some methods of speeding up the convergence of iteration methods
- Heavy Ball Method — fmin.xyz
- Jang & Ryu (2025) — Point Convergence of Nesterov's Accelerated Gradient Method
- Ryu (2026) — Nesterov Flow May Travel Infinitely Long to Converge to a Minimizer
- Cohen et al. (2025) — Understanding Optimization in Deep Learning with Central Flows
- Regis & Chewi (2026) — Rod Flow: A Continuous-Time Model for GD at the Edge of Stability
- Stochastic gradient algorithms from ODE splitting perspective
- Introduction to Gradient Flows in the 2-Wasserstein Space
- Jordan, Kinderlehrer, Otto (1998) — The Variational Formulation of the Fokker–Planck Equation
- OSQP: Stellato, Banjac, Goulart, Bemporad, Boyd, *Math. Prog. Comp.* 12(4), 2020 — osqp.org
- SCS: O'Donoghue, Chu, Parikh, Boyd, *JOTA* 169(3), 2016 — cvxgrp.org/scs



- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2016)
- Andersen Ang — Convergence of Gradient Flow
- Francis Bach — Gradient flows (blog)
- Francis Bach (2021) — Continuized Acceleration
- Off Convex Path — Gradient flow и gradient descent
- Polyak (1964) — Some methods of speeding up the convergence of iteration methods
- Heavy Ball Method — fmin.xyz
- Jang & Ryu (2025) — Point Convergence of Nesterov's Accelerated Gradient Method
- Ryu (2026) — Nesterov Flow May Travel Infinitely Long to Converge to a Minimizer
- Cohen et al. (2025) — Understanding Optimization in Deep Learning with Central Flows
- Regis & Chewi (2026) — Rod Flow: A Continuous-Time Model for GD at the Edge of Stability
- Stochastic gradient algorithms from ODE splitting perspective
- Introduction to Gradient Flows in the 2-Wasserstein Space
- Jordan, Kinderlehrer, Otto (1998) — The Variational Formulation of the Fokker–Planck Equation
- OSQP: Stellato, Banjac, Goulart, Bemporad, Boyd, *Math. Prog. Comp.* 12(4), 2020 — osqp.org
- SCS: O'Donoghue, Chu, Parikh, Boyd, *JOTA* 169(3), 2016 — cvxgrp.org/scs
- Boyd, Parikh, Chu, Peleato, Eckstein — Distributed Optimization via ADMM (Found. Trends ML, 2011)